

School Of Computing and Informatics

Database Programming

ICU Beds & Hospital Capacity Management

Submitted To: Dr. Samir Tartir

Section Number: 1

Summer – 2026

Table of Contents

Table of Contents 2
Part 1: Relational Database Design, & Views 5
1.1 Requirements & ERD 5
1.1.1 Problem Description 5
1.1.2 Database Design 6
Conceptual Design 6
Mapping 6
Logical Design 7
Physical Design 8
1.1.3 Oracle DDL Creation Statements 8
1.2 Security Layer 10
1.2.1 First View 10
SQL Code 10
Test Insert / Update 11
1.2.2 Second View 12
SQL Code 12
Test Insert / Update 12
1.3 Optimizing the Performance 13
1.3.1 Common Summary Queries 13
1.3.2 Materialized View 15
SQL Code 15
Test Changes on the Base Table 16
1.4 Evaluation 16
1.4.1 How Updatable Views Improve Database Security 16
1.4.2 Materialized Views Versus Regular Queries 18
Part 2: PL/SQL Logic, Triggers & Automated Auditing 19
2.1 PL/SQL Procedures & Functions 19
2.1.1 Procedure 19
Procedure PL/SQL Code 19

Executing The Procedure 20
2.1.2 Function 20
Function PL/SQL Code 20
Executing The Function 21
2.2 PL/SQL Triggers 21
2.2.1 First Trigger 21
Trigger PL/SQL Code 21
Test Insert 22
Test Update 23
Test Delete 23
2.2.2 Second Trigger 23
Trigger PL/SQL Code 24
Test Insert 24
Test Invalid Data 25
2.2.3 Third Trigger 25
Trigger PL/SQL Code 25
Test Update 26
2.2.4 Fourth Trigger 26
Trigger PL/SQL Code 27
Test Invalid Data 28
Test Insert 28
2.2.5 Fifth Trigger 28
Trigger PL/SQL Code 28
Test Delete/Update 29
2.3 PL/SQL Package 29
2.3.1 Audit Log Table 29
2.3.2 Package PL/SQL Code 29
Specification: 29
Body: 30
2.3.3 Testing 32
Testing The Log 32
Testing Bill Count Function 32
2.4 Evaluation 33

Part 3: Multi-Model NoSQL Implementation 35
3.1 Document Database 35
3.1.1 How Did I Export Data From Oracle 35
3.1.2 Collection 36
3.1.3 Adding Documents 36
Small Sample of the Inserted Documents 36
3.1.4 Find Operations 37
Count 37
Find a Specific Stay 37
Find Stays in Order 38
3.2 Graph Database 38
3.2.1 How Did I Export Data from Oracle 39
3.2.2 Creating Hospital Nodes 40
3.2.3 Creating Hospital Transfers Relationships 41
3.2.4 Creating the Routes Between Hospitals 41
3.2.5 Cypher Queries 42
Hospital Transfer Graph 42
Hospital Routes Graph 42
Finding the Shortest Path Between Two Hospitals 43
Finding Outgoing Transfers from a Specific Hospital 43
Finding Incoming Transfers from a Specific Hospital 43
Number of Transfers 44
Number of Routes 44
Finding Transfers on a Specific Date 44
Schema Visualization 45
3.3 Performance Benchmarking 45
3.3.1 MongoDB VS Oracle 45
MongoDB 45
Oracle 46
Comparison 47
3.3.2 Neo4j VS Oracle 47
Neo4j 47
Oracle 48

Comparison 49
3.3.3 Overall Performance 49
Part 4: Application Integration, Testing & System 49
4.1 Software Application 49
4.1.1 Admin 50
4.1.2 User 58
4.1.3 Running Core Transactions 62
Package Usage 62
Procedure Usage 62
View Usage 62
Materialized View Usage 63
4.2 Testing 63
4.2.1 Successful Run Test 63
Test 1: Adding Medical Information to a Stay 63
Test 2: Completing a Transfer 64
4.2.2 Invalid Input Test 65
Test 1: Adding Invalid Daily Charge 65
4.2.3 Error Test 65
Test 1: MongoDB Connection Error 65
4.3 System Architecture Evaluation 66
References 68

Part 1: Relational Database Design, & Views

1.1 Requirements & ERD

1.1.1 Problem Description

The problem is that managing ICU operations are difficult to manage across different hospitals because hospitals need to keep track of ICU bed availability, specialist equipment allocations such as mechanical ventilators, patient stays, daily stay charges, patient information, and inter-hospital transfers, and if these data are not correctly managed, it becomes harder to know which beds are available, which equipment are available, which patient has a stay and using equipment's, and what happened during each stay. Therefore, we will introduce the ICU Beds & Hospital Capacity Management

database, as it will be designed to connect all of the information stated in one system including hospitals, ICU beds, patients, ICU stays, equipment allocations, daily charges, and hospital transfers, which makes it easier to real-time track and manage all ICU operations. The ICU Beds & Hospital Capacity Management database will help keeping accurate records and provide a clear view on all ICU operations and without it, problems might appear such as assigning ICU bed or equipment that is already in use, losing history of stays, patients, transfers, and bills, or having wrong information about the ICU stays and capacity.

1.1.2 Database Design

Conceptual Design

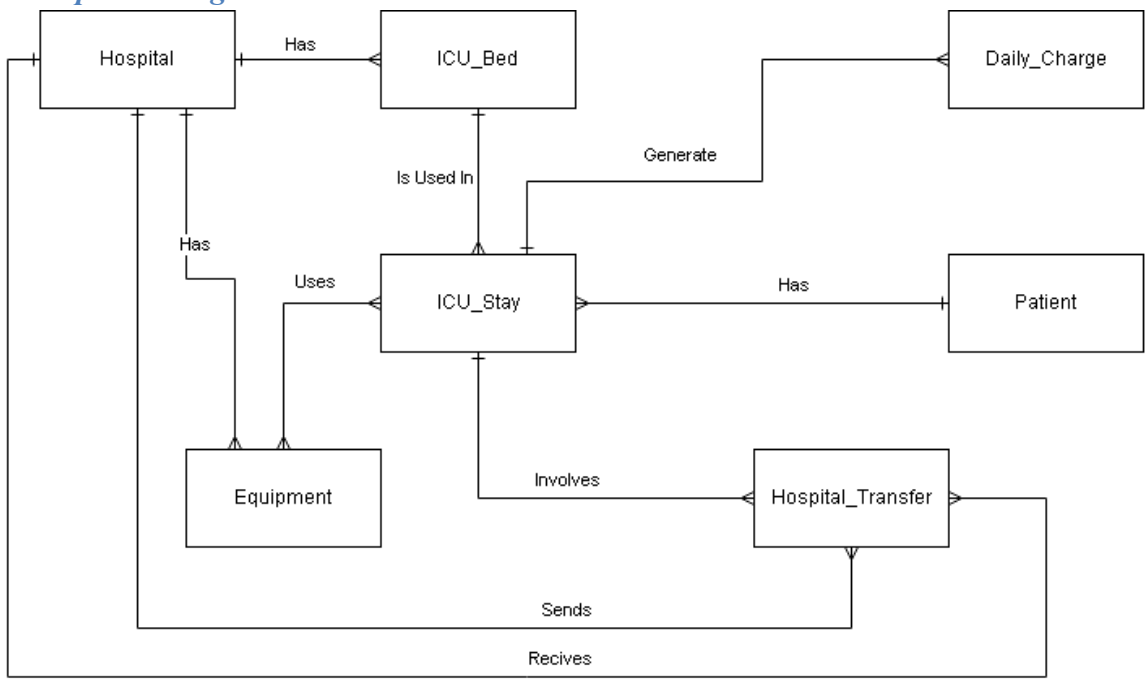


Figure 1: Conceptual Design

Mapping

Before Mapping:

- Hospital(hospitalId, hospitalName, location, contactNumber, email)
- ICU_Bed(bedId, bedNumber, status)
- ICU_Stay(stayId, enteringDate, leavingDate)
- Patient(patientId, name, dateOfBirth, gender, nationalId , phoneNumber, email)
- Daily_Charge(chargeId, chargeDate, amount)
- Equipment(equipmentId, equipmentName, status)
- Hospital_Transfer(transferId, transferDate)

After Mapping:

- Hospital(**hospitalId**, hospitalName, country, city, street, contactNumber, email)
- ICU_Bed(**bedId**, bedNumber, status, *hospitalId*)
- ICU_Stay(**stayId**, enteringDate, leavingDate, *patientId, bedId*)
- Patient(**patientId**, Fname, Mname, Lname, dateOfBirth, gender, nationalId)
- Patient_Phone(***patientId*, phoneNumber**)
- Patient_Email(***patientId*, email**)
- Daily_Charge(**chargeId**, chargeDate, amount, *stayId*)
- Equipment(**equipmentId**, equipmentName, status, *hospitalId*)
- Equipment_Stay(***stayId*, *equipmentId***)
- Hospital_Transfer(**transferId**, transferDate, *stayId*, *fromhospitalId, tohospitalId*)

Logical Design

For a better view, the diagram can be accessed through this link: Logical Design

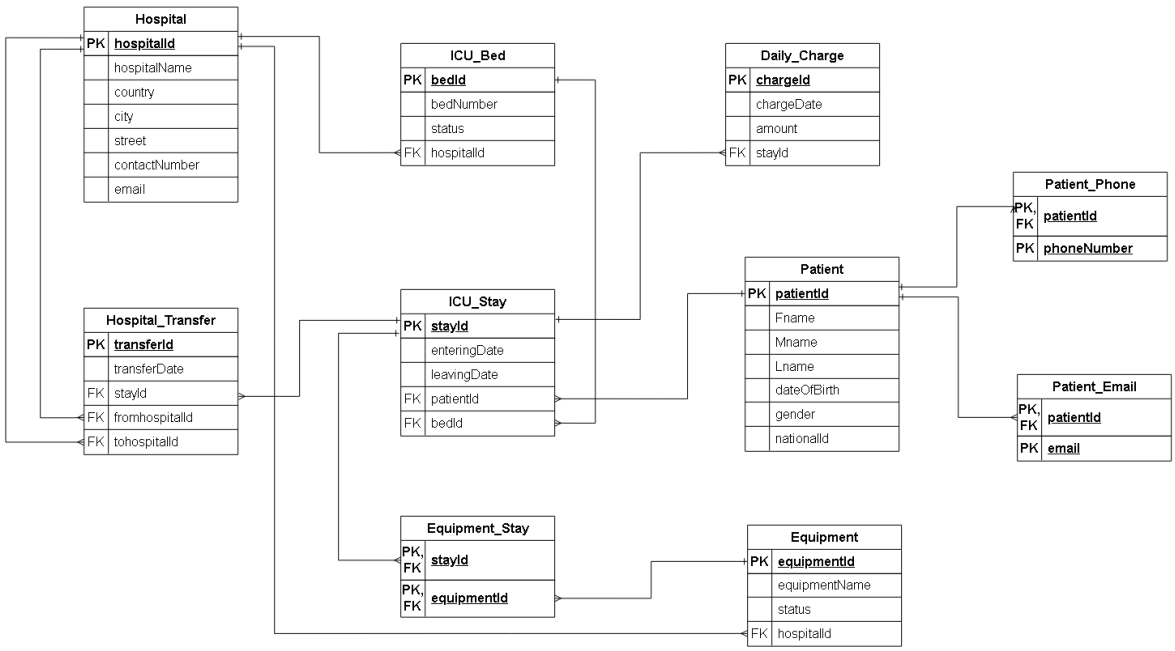


Figure 2: Logical Design

Physical Design

For a better view, the diagram can be accessed through this link: [Physical Design]

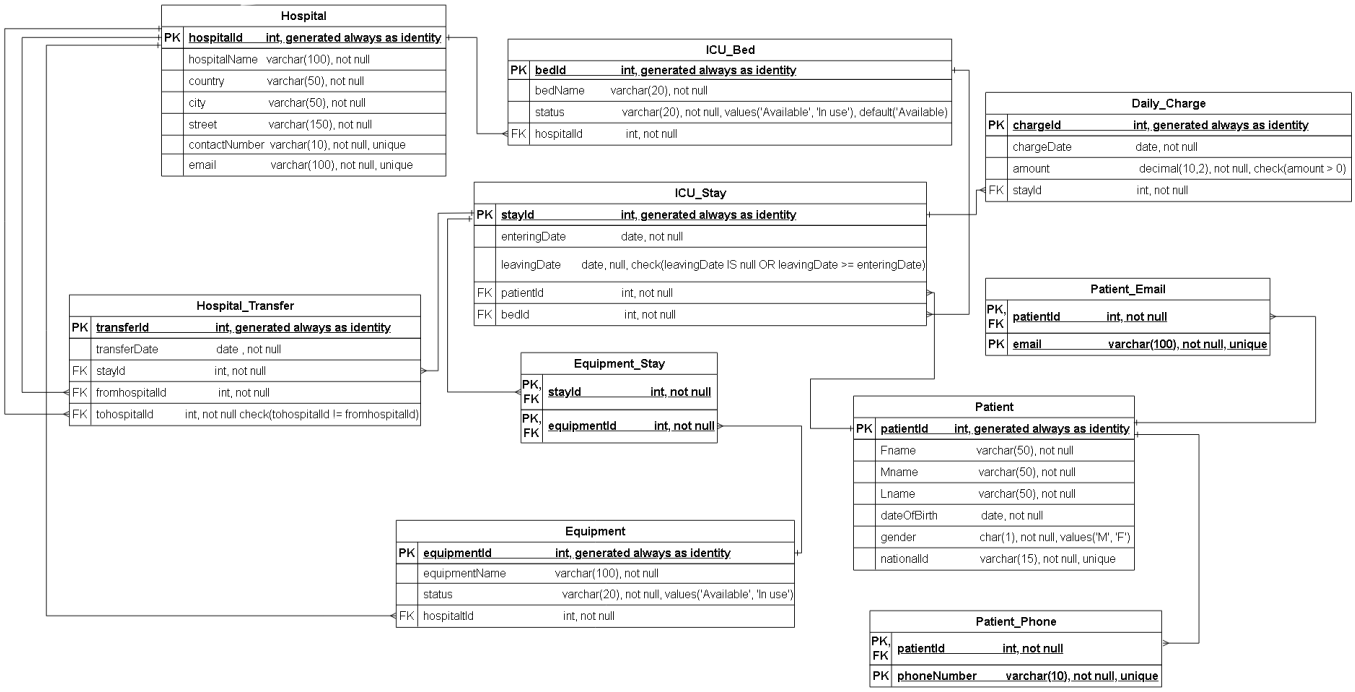


Figure 3: Physical Design

The composite unique constraints such as UNIQUE (hospitalId, bedNumber), UNIQUE (hospitalId, equipmentName) and UNIQUE (stayId, chargeDate) were not displayed in the physical diagram to maintain its clarity. However, they were added in the actual Oracle ICU_Bed, Equipment and Daily_Charge tables.

1.1.3 Oracle DDL Creation Statements

```
CREATE TABLE Hospital(
hospitalId INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
hospitalName VARCHAR(100) NOT NULL,
country VARCHAR(50) NOT NULL,
city VARCHAR(50) NOT NULL,
street VARCHAR(150) NOT NULL,
contactNumber VARCHAR(10) NOT NULL UNIQUE,
email VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE ICU_Bed(
bedId INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
bedNumber VARCHAR(20) NOT NULL,
status VARCHAR(20) DEFAULT ‘Available’ NOT NULL,
hospitalId INT NOT NULL,
CHECK(status IN(‘Available’, ‘In use’)),
UNIQUE (hospitalId, bedNumber),
CONSTRAINT “Hospital-ICUBed_FK” FOREIGN KEY(hospitalId)
```

```
REFERENCES Hospital(hospitalId) ON DELETE CASCADE
);

CREATE TABLE Equipment(
equipmentId INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
equipmentName VARCHAR(100) NOT NULL,
status VARCHAR(20) NOT NULL,
hospitalId INT NOT NULL,
CHECK(status IN('Available', 'In use')),
UNIQUE (hospitalId, equipmentName),
CONSTRAINT "Hospital-Equipment_FK" FOREIGN KEY(hospitalId)
REFERENCES Hospital(hospitalId) ON DELETE CASCADE
);

CREATE TABLE Patient(
patientId INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
Fname VARCHAR(50) NOT NULL,
Mname VARCHAR(50) NOT NULL,
Lname VARCHAR(50) NOT NULL,
dateOfBirth DATE NOT NULL,
gender CHAR(1) NOT NULL,
nationalId VARCHAR(15) NOT NULL UNIQUE,
CHECK(gender IN('M', 'F'))
);

CREATE TABLE Patient_Email(
patientId INT NOT NULL,
email VARCHAR(100) NOT NULL UNIQUE,
PRIMARY KEY(patientId, email),
CONSTRAINT "Patient-Email_FK" FOREIGN KEY(patientId)
REFERENCES Patient(patientId) ON DELETE CASCADE
);

CREATE TABLE Patient_Phone(
patientId INT NOT NULL,
phoneNumber VARCHAR(10) NOT NULL UNIQUE,
PRIMARY KEY(patientId, phoneNumber),
CONSTRAINT "Patient-Phone_FK" FOREIGN KEY(patientId)
REFERENCES Patient(patientId) ON DELETE CASCADE
);

CREATE TABLE ICU_Stay(
stayId INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
enteringDate DATE NOT NULL,
leavingDate DATE NULL,
patientId INT NOT NULL,
bedId INT NOT NULL,
CHECK(leavingDate IS NULL OR leavingDate >= enteringDate),
CONSTRAINT "Patient-ICU_Stay_FK" FOREIGN KEY(patientId)
REFERENCES Patient(patientId) ON DELETE CASCADE,
CONSTRAINT "ICU_Bed-ICU_Stay_FK" FOREIGN KEY(bedId)
REFERENCES ICU_Bed(bedId) ON DELETE CASCADE
);

CREATE TABLE Equipment_Stay(
stayId INT NOT NULL,
equipmentId INT NOT NULL,
PRIMARY KEY(stayId, equipmentId),
CONSTRAINT "E-Equipment_Stay_FK" FOREIGN KEY(equipmentId)
REFERENCES Equipment(equipmentId)ON DELETE CASCADE,
CONSTRAINT "S-Equipment_Stay_FK" FOREIGN KEY(stayId)
```

```
REFERENCES ICU_Stay(stayId) ON DELETE CASCADE
);

CREATE TABLE Daily_Charge(
chargeId INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
chargeDate DATE NOT NULL,
amount DECIMAL(10,2) NOT NULL,
stayId INT NOT NULL,
CHECK(amount > 0),
UNIQUE (stayId, chargeDate),
CONSTRAINT “ICU_Stay-Daily_Charge_FK” FOREIGN KEY(stayId)
REFERENCES ICU_Stay(stayId) ON DELETE CASCADE
);

CREATE TABLE Hospital_Transfer(
transferId INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
transferDate DATE NOT NULL,
stayId INT NOT NULL,
fromhospitalId INT NOT NULL,
tohospitalId INT NOT NULL,
CHECK(fromhospitalId != tohospitalId),
CONSTRAINT “ICU_Stay-Hospital_Transfer_FK” FOREIGN KEY(stayId)
REFERENCES ICU_Stay(stayId) ON DELETE CASCADE,
CONSTRAINT “HospitalA-Hospital_Transfer_FK” FOREIGN KEY(fromhospitalId)
REFERENCES Hospital(hospitalId) ON DELETE CASCADE,
CONSTRAINT “HospitalB-Hospital_Transfer_FK” FOREIGN KEY(tohospitalId)
REFERENCES Hospital(hospitalId) ON DELETE CASCADE
);
```

1.2 Security Layer

1.2.1 First View

The first view I created is a view that allows you to access and manage ICU beds that the hospital 554 (Aqaba Medical Center 094) has, so Aqaba Medical Center 094 staff can only access the ICU beds for Aqaba Medical Center 094 instead of accessing all ICU bed records.

SQL Code

```
CREATE VIEW hospitalBeds_view AS
SELECT bedId, bedNumber, status, hospitalId
FROM ICU_BED
WHERE hospitalId = 554
WITH CHECK OPTION;
```

	BEDID	BEDNUMBER	STATUS	HOSPITALID
1	2131	ICU-01	In use	554
2	2132	ICU-02	In use	554
3	2133	ICU-03	In use	554
4	2134	ICU-04	Available	554
5	2135	ICU-05	Available	554
6	2136	ICU-06	Available	554
7	2137	ICU-07	Available	554
8	2138	ICU-08	Available	554
9	2139	ICU-09	Available	554
10	2140	ICU-10	Available	554

Figure 4: Select Statement Result

Test Insert / Update

```
INSERT INTO hospitalBeds_view(bedNumber, status, hospitalId)
VALUES('ICU-11', 'Available', 554);
```

	BEDID	BEDNUMBER	STATUS	HOSPITALID
1	2131	ICU-01	In use	554
2	2132	ICU-02	In use	554
3	2133	ICU-03	In use	554
4	2134	ICU-04	Available	554
5	2135	ICU-05	Available	554
6	2136	ICU-06	Available	554
7	2137	ICU-07	Available	554
8	2138	ICU-08	Available	554
9	2139	ICU-09	Available	554
10	2140	ICU-10	Available	554
11	2202	ICU-11	Available	554

Figure 5: Result of Inserting into The View

As we can see in figure 5 the row with bedId = 2202 was inserted successfully through the view with bedNumber = ICU-11.

```
UPDATE hospitalBeds_view
SET bedNumber = 'ICU-12'
WHERE bedNumber = 'ICU-11';
```

	BEDID	BEDNUMBER	STATUS	HOSPITALID
1	2131	ICU-01	In use	554
2	2132	ICU-02	In use	554
3	2133	ICU-03	In use	554
4	2134	ICU-04	Available	554
5	2135	ICU-05	Available	554
6	2136	ICU-06	Available	554
7	2137	ICU-07	Available	554
8	2138	ICU-08	Available	554
9	2139	ICU-09	Available	554
10	2140	ICU-10	Available	554
11	2202	ICU-12	Available	554

Figure 6: Result of Updating a Row in The View

As we can see in figure 6, before the update, the row with bedId = 2202 had the bedNumber = ICU-11, but after executing the update statement through the view, the bedNumber has changed to ICU-12.

1.2.2 Second View

The second view I created is a view that allows you to access and manage only the transfers that originate from the hospital 554 (Aqaba Medical Center 094), so Aqaba Medical Center 094 staff can only access and manage the transfers that originate from Aqaba Medical Center 094 instead of accessing all transfer records.

SQL Code

```
CREATE VIEW hospitalTransfers_View AS
SELECT transferId, transferDate, stayId, fromhospitalId, tohospitalId
FROM Hospital_Transfer
WHERE fromhospitalId = 554
WITH CHECK OPTION;
```

	TRANSFERID	TRANSFERDATE	STAYID	FROMHOSPITALID	TOHOSPITALID
1	673	08-SEP-25	11765	554	555
2	674	28-SEP-25	11767	554	555
3	675	18-OCT-25	11769	554	555
4	676	07-NOV-25	11771	554	555

Figure 7: Select Statement Result

Test Insert / Update

stayId = 9369 is a stay that is associated with hospital that has the id 554.

```
INSERT INTO hospitalTransfers_View(transferDate, stayId, fromhospitalId,
tohospitalId)
VALUES(CURRENT_DATE, 9369, 554, 555);
```

	TRANSFERID	TRANSFERDATE	STAYID	FROMHOSPITALID	TOHOSPITALID
1	673	08-SEP-25	11765	554	555
2	674	28-SEP-25	11767	554	555
3	675	18-OCT-25	11769	554	555
4	676	07-NOV-25	11771	554	555
5	701	22-AUG-26	9369	554	555

Figure 8 :Result of Inserting into The View

As we can see in figure 8, the transferId = 701 wasn't there before the insert, but after executing the insert statement through the view, a new transfer was successfully created.

```
UPDATE hospitalTransfers_View
SET transferDate = '21-AUG-26'
WHERE stayId = 9369;
```

	TRANSFERID	TRANSFERDATE	STAYID	FROMHOSPITALID	TOHOSPITALID
1	673	08-SEP-25	11765	554	555
2	674	28-SEP-25	11767	554	555
3	675	18-OCT-25	11769	554	555
4	676	07-NOV-25	11771	554	555
5	701	21-AUG-26	9369	554	555

Figure 9: Result of Updating a Row in The View

As we can see in figure 9, the transfer date was "22- AUG-26" for transferId = 701, but after executing the update statement through the view, the transfer date was changed successfully to "21-AUG-26".

1.3 Optimizing the Performance

1.3.1 Common Summary Queries

The first summary query is a query that retrieves the ICU capacity of beds per hospital and specify how many available and in use beds each hospital has.

```
SELECT b.hospitalId, h.hospitalName, status, COUNT(b.bedId)
FROM ICU_Bed b
INNER JOIN Hospital h
ON h.hospitalId = b.hospitalId
GROUP BY b.hospitalId, h.hospitalName, b.status
ORDER BY b.hospitalId ASC;
```

	HOSPITALID	HOSPITALNAME	STATUS	COUNT(B.BEDID)
1	461	Amman Medical Center 001	Available	7
2	461	Amman Medical Center 001	In use	3
3	462	Irbid Medical Center 002	Available	7
4	462	Irbid Medical Center 002	In use	3
5	463	Zarqa Medical Center 003	Available	7
6	463	Zarqa Medical Center 003	In use	3
7	464	Aqaba Medical Center 004	Available	7
8	464	Aqaba Medical Center 004	In use	3
9	465	Salt Medical Center 005	Available	7
10	465	Salt Medical Center 005	In use	3
11	466	Madaba Medical Center 006	Available	7
12	466	Madaba Medical Center 006	In use	3
13	467	Jerash Medical Center 007	Available	7
14	467	Jerash Medical Center 007	In use	3
15	468	Karak Medical Center 008	Available	7
16	468	Karak Medical Center 008	In use	3
17	469	Mafraq Medical Center 009	Available	7
18	469	Mafraq Medical Center 009	In use	3

Figure 10: Small Sample of Result

The second summary query is a query that retrieves all the hospitals in the systems and calculates the total charges for each hospital.

```
SELECT h.hospitalId, h.hospitalName, SUM(d.amount)
FROM Hospital h
INNER JOIN ICU_Bed b
ON h.hospitalId = b.hospitalId
INNER JOIN ICU_Stay s
ON b.bedId = s.bedId
INNER JOIN Daily_Charge d
ON s.stayId = d.stayId
GROUP BY h.hospitalId, h.hospitalName
ORDER BY h.hospitalId ASC;
```

	HOSPITALID	HOSPITALNAME	SUM(D.AMOUNT)
1	461	Amman Medical Center 001	119796.66
2	462	Irbid Medical Center 002	117640.3
3	463	Zarqa Medical Center 003	115995.36
4	464	Aqaba Medical Center 004	122910.64
5	465	Salt Medical Center 005	125821.14
6	466	Madaba Medical Center 006	118263.86
7	467	Jerash Medical Center 007	115466.3
8	468	Karak Medical Center 008	120682.16
9	469	Mafraq Medical Center 009	126208.24
10	470	Tafilah Medical Center 010	120560.54
11	471	Amman Medical Center 011	117403.06
12	472	Irbid Medical Center 012	119152.3
13	473	Zarqa Medical Center 013	124626.96
14	474	Aqaba Medical Center 014	120511.84
15	475	Salt Medical Center 015	119541.94
16	476	Madaba Medical Center 016	121655.26
17	477	Jerash Medical Center 017	121928.3
18	478	Karak Medical Center 018	119963.76

Figure 11: Small Sample of Result

The third summary query is a query that retrieves all the patients in the systems and calculates how many stays they have.

```
SELECT p.patientId, p.Fname, p.Lname , COUNT(s.stayId)
FROM Patient p
```

```
INNER JOIN ICU_Stay s
ON p.patientId = s.patientId
GROUP BY p.patientId, p.Fname, p.Lname
ORDER BY COUNT(s.stayId) DESC;
```

	PATIENTID	FNAME	LNAME	COUNT(S.STAYID)
1	5522	Ahmad	Haddad	2
2	5530	Noor	Abdallah	2
3	5532	Ahmad	Haddad	2
4	5535	Yousef	Odeh	2
5	5536	Dana	Mansour	2
6	5537	Khaled	Hamdan	2
7	5556	Dana	Mansour	2
8	5563	Sara	Nasser	2
9	5565	Yousef	Odeh	2
10	5573	Sara	Nasser	2
11	5594	Lina	Saleh	2
12	5610	Noor	Abdallah	2
13	5620	Noor	Abdallah	2
14	5624	Lina	Saleh	2
15	5643	Sara	Nasser	2

Figure 12: Small Sample of Result

1.3.2 Materialized View

I created a materialized view based on the second summary query which calculates the total charges for each hospital. Executing this summary query repeatedly needs more processing and recalculating every time the query is executed but the materialized view stores the results of the query which speed up the retrieval process without recalculating every time the view is executed.

I used BUILD IMMEDIATE because I wanted the materialized view to be filled as soon as its created. I also used REFRESH COMPLETE because I wanted the whole materialized view to be recalculated when it is refreshed. I also used ON COMMIT because I wanted the materialized view to be automatically refreshed instead of refreshing it manually when a change is committed on the base table.

SQL Code

```
CREATE MATERIALIZED VIEW hospitalCharges_MatView BUILD IMMEDIATE REFRESH COMPLETE ON
COMMIT AS
SELECT h.hospitalId, h.hospitalName, SUM(d.amount)
FROM Hospital h
INNER JOIN ICU_Bed b
ON h.hospitalId = b.hospitalId
INNER JOIN ICU_Stay s
ON b.bedId = s.bedId
INNER JOIN Daily_Charge d
ON s.stayId = d.stayId
GROUP BY h.hospitalId, h.hospitalName
ORDER BY h.hospitalId ASC;
```

HOSPI...	HOSPITALNAME	SUM(D.AMOUNT)
461	Amman Medical Center 001	119796.66
462	Irbid Medical Center 002	117640.3
463	Zarqa Medical Center 003	115995.36
464	Aqaba Medical Center 004	122910.64
465	Salt Medical Center 005	125821.14
466	Madaba Medical Center 006	118263.86
467	Jerash Medical Center 007	115466.3
468	Karak Medical Center 008	120682.16
469	Mafraq Medical Center 009	126208.24
470	Tafilah Medical Center 010	120560.54
471	Amman Medical Center 011	117403.06

Figure 13: Select Statement Result (Small Sample)

Test Changes on the Base Table

chargeId = 33401 is associated with Amman Medical Center 001 that has the id 461.

```
UPDATE Daily_Charge
SET amount = amount + 1000
WHERE chargeId = 33401;
COMMIT;
```

HOSPI...	HOSPITALNAME	SUM(D.AMOUNT)
461	Amman Medical Center 001	120796.66

Figure 14: Select Statement Result

As we can see in figure 14, the total charge before the update was 119796.66. After updating a charge in the daily charge table by 1000 and committing the change, the total charge in the materialized view for Amman Medical Center 001 also changed and became 120796.66

1.4 Evaluation

1.4.1 How Updatable Views Improve Database Security

A view is a virtual table that is based on a SELECT statement. Updatable views highly contribute to the database security improvement because updatable views allow users access and edit specific part of the data without giving them direct access on base table. Therefor, users can only use the data that they only need instead of having access to all the data stored in the tables as the view can hide some of highly sensitive data that not everyone can see such as national id.

For example, a bank company can have some sensitive data in their tables such as customers account balances, national ids, and credit card information and at the same time, that table has data that some employees need such as customer names and contact information, so instead of giving these employees access to the whole table, views can be

created to show only the rows or columns that these employees need which reduces unauthorized access to data and guarantee data confidentiality and integrity.

Another thing is that updatable views allow users to make changes such as an INSERT or UPDATE operation without accessing the base table directly, as WITH CHECK OPTION provide control over these changes because they prevent the user from inserting or updating a record that does not match the condition of the view, for example, a manager want to insert a new employee in his department, so he use an updatable view to insert a record only in his department if the view uses WITH CHECK OPTION, and cannot insert a new employee in another department which prevent unauthorized changes and helps guarantee data integrity.

In my database, I applied those concepts using updatable views, the hospitalBeds view only shows the ICU beds that belong to hospital 554 which means that the hospital staff can access and manage their ICU beds that belongs only to their hospital. The hospitalTransfers view only shows the transfers that originate from hospital 554, so hospital staff can manage their own transfer records instead of having access to all hospital transfer in the system. Also, both views use WITH CHECK OPTION to prevent users to make changes outside the condition of the view.

On the other hand, updatable views are not the complete security solution because the database permissions are also important because if users have direct permissions to the original tables, they may bypass the view completely and actually be able to access the base table.

Also, if we want to make separate views for everything in the system and for different users, hospitals, or departments, it will be very hard to manage, for example, the views I created were directly for a specific hospital which is hospital 554, and if we have 1000 hospital, we may have to create one view for each hospital which is 1000 in total.

Another thing is that if we want to create a view using more than one table, the view will contain joins between these tables which will make the view not always updatable, but in some cases, simple joins might still work. If the view cannot be updated directly, then an INSTEAD OF trigger can be used to define what should happen when an INSERT, UPDATE, or DELETE operation happens which makes it updatable.

Also, if a major change happens to the base table, the view may also need to be changed or if the columns provided to the view were changed or removed, the view may also be changed, so changes to the base table must be maintained carefully.

So, overall, updatable views improve database security in my system, hospital staff only interact with views and can’t see or change record belong to other hospital, but these

views on their own are not a full solution, as permission on the base tables should be managed carefully, and they also come with a real cost which is one view per hospital.

1.4.2 Materialized Views Versus Regular Queries

Both of the materialized views and regular queries used to retrieve data, but they differ in terms of performance because materialized views store the result of a SELECT statement physically, but a regular query processes the SELECT statement every time it is executed.

One of the main advantages of materialized views is the query response time, especially when dealing with large data and repeated aggregated query, for example, executing an aggregated query on a table that contains more than 7 million records may take around 5 seconds but if we use materialized view for the same result, it takes around 0.01 seconds which is around 500 times faster than executing regular query.

But this performance improvement comes with a cost because the stored result in the materialized view must be maintained when the data in the base table changes, if the base table is updated and the materialized view is not refreshed, the result stored in the materialized view may become outdated. On the other hand, a regular query processes the current recent data from the base table when it is executed so it does not require refreshment.

The refresh method affects the performance of the materialized view, as the refreshment method differs from one materialized view from another, as REFRESH COMPLETE executes the whole query again and rebuilds the materialized view, REFRESH FAST only updates the changes but requires logs for the materialized view. COMPLETE has lower speed and higher resource usage, while FAST has higher speed and lower resource usage. Also, ON COMMIT automatically updates the materialized view when the base table changes are committed, but it comes with a high load on the base tables.

Materialized view reduce the processing needed when retrieving the same calculated result many times, but some of this processing is moved to the refresh method, while regular queries do not have this refresh cost but all of the joins and calculation in the query must be processed again when the query is executed.

On the other hand, regular queries have their own advantages and disadvantages, they do not store the result, so they do not need any refresh process when the base table changes. Also, every time the query is executed, it reads the data directly from the base table and processes everything again which means that regular queries are useful when the data changes a lot.

In my database, the materialized view I created uses REFRESH COMPLETE ON COMMIT which means every time a change is committed on Hospital, ICU_Bed,

ICU_stay, or Daily_Charge tables, the materialized view is rebuilt from scratch which keep the materialized view updated, but it also means that updating only a simple small row in one table will also makes the materialized view to rebuild and recalculate the entire materialized view. If I used REFRESH FAST, only the affected rows would be recalculated but it requires a log set up first which REFRESH COMPLETE does not require.

Part 2: PL/SQL Logic, Triggers & Automated Auditing

2.1 PL/SQL Procedures & Functions

2.1.1 Procedure

I created a procedure that end and ICU stay through updating its leaving date. An active ICU stay have a NULL leaving date. The procedure has two input parameters (stay id, and leaving date), then the procedure check if that stay has already finished by checking if the leaving date is NULL or NOT NULL, and it also check if the entered leaving date is equal or later than the entering date before updating the record.

Procedure PL/SQL Code

```
CREATE OR REPLACE PROCEDURE finishStay_PRC(
stayIdParameter  IN INT,
leavingDateParameter IN DATE
) AS

enteringDateVar DATE;
leavingDateVar DATE;

BEGIN

SELECT enteringDate, leavingDate
INTO enteringDateVar, leavingDateVar
FROM ICU_Stay
WHERE stayId = stayIdParameter;

IF leavingDateVar IS NULL AND leavingDateParameter >= enteringDateVar THEN
UPDATE ICU_Stay
SET leavingDate = leavingDateParameter
WHERE stayId = stayIdParameter;
DBMS_OUTPUT.PUT_LINE('ICU Stay With Id = ' || stayIdParameter || 'Has Been Finished
Successfully');

ELSE
RAISE_APPLICATION_ERROR(-20999, 'An Error Has Occured (e.g., Invalid Date, Stay Has
Finished)');

END IF;

EXCEPTION
WHEN NO_DATA_FOUND THEN
DBMS_OUTPUT.PUT_LINE('ICU Stay Does Not Exist');
```

```
END finishStay_PRC;
```

Executing The Procedure

```
BEGIN
finishStay_PRC(11821, CURRENT_DATE);
END;
```

```
ICU Stay With Id = 11821Has Been Finished Successfully


PL/SQL procedure successfully completed.
```

Figure 15: Result of Executing

2.1.2 Function

Since the Daily_Charge table stores daily charges for ICU stays, I created a function to calculate the total charges for a specific ICU stay. The function has one input parameter (stayId) and it retrieves all daily charges connected with that ICU stay, then it calculates the sum and returns the results. The functions check if there are no charges connected with that stay and raise an application error.

Function PL/SQL Code

```
CREATE OR REPLACE FUNCTION getStayTotal_FUNC(
stayIdParameter IN INT
)
RETURN DECIMAL AS

totalChargesPerStay DECIMAL(10,2);

BEGIN

SELECT SUM(amount)
INTO totalChargesPerStay
FROM Daily_Charge
WHERE stayId = stayIdParameter;

IF totalChargesPerStay IS NULL THEN
RAISE_APPLICATION_ERROR(-20998, 'The Stay Has No Charges Yet');
ELSE
DBMS_OUTPUT.PUT_LINE('Total Charges For The Stay With Id = ' || stayIdParameter || '
Is: ' || totalChargesPerStay);
RETURN totalChargesPerStay;
END IF;

END getStayTotal_FUNC;
```

Executing The Function

```
DECLARE
total DECIMAL(10,2);
BEGIN
total := getStayTotal_FUNC(8036);
END;
```

```
Total Charges For The Stay With Id = 8036 Is: 2507.44


PL/SQL procedure successfully completed.
```

Figure 16: Result of Executing

2.2 PL/SQL Triggers

While implementing the database, I found out that only three triggers were not enough to enforce the most critical business rules, so I implemented more than three trigger to handle and enforce these rules which help in maintaining data integrity in the database.

2.2.1 First Trigger

I created a row level trigger that manages the availability of ICU beds, and it is executed before an operation such as insert or update or delete on ICU_Stay table. ICU beds are connected with ICU stays, so when a new ICU stay is inserted, the trigger check the status of the selected bed, if its available, the status of the bed is changed to in use, but if the selected bed status is in use, then the trigger raise an error and it does not create the ICU stay. I also handled the updating case, as to finish an ICU stay, we must update its leaving date, so when we update the leaving date, the assigned bed status is changed to available. I also prevented users from deleting an ICU stay.

Trigger PL/SQL Code

```
CREATE OR REPLACE TRIGGER bedStatus_TRG
BEFORE INSERT OR UPDATE OR DELETE ON ICU_Stay
FOR EACH ROW

DECLARE
bedStatusVar VARCHAR(20);

BEGIN

IF INSERTING THEN

IF :NEW.leavingDate IS NULL THEN

SELECT status
INTO bedStatusVar
FROM ICU_Bed
```

```
WHERE bedId = :NEW.bedId;

IF bedStatusVar = 'Available' THEN

UPDATE ICU_Bed
SET status = 'In use'
WHERE bedId = :NEW.bedId;

ELSE
RAISE_APPLICATION_ERROR(-20998, 'The Bed is Already Reserved');
END IF;
END IF;
END IF;

IF UPDATING THEN

IF :OLD.leavingDate IS NULL AND :NEW.leavingDate IS NOT NULL THEN

UPDATE ICU_Bed
SET status = 'Available'
WHERE bedId = :OLD.bedId;

ELSE
RAISE_APPLICATION_ERROR(-20997, 'ICU Stay Already Finished');

END IF;
END IF;

IF DELETING THEN
RAISE_APPLICATION_ERROR(-20996, 'You Cant Delete an ICU Stay');
END IF;

END bedStatus_TRG;
```

Test Insert

I tried to insert an ICU stay with a bed that is already in use, but the trigger prevented me because the bed status = In use.

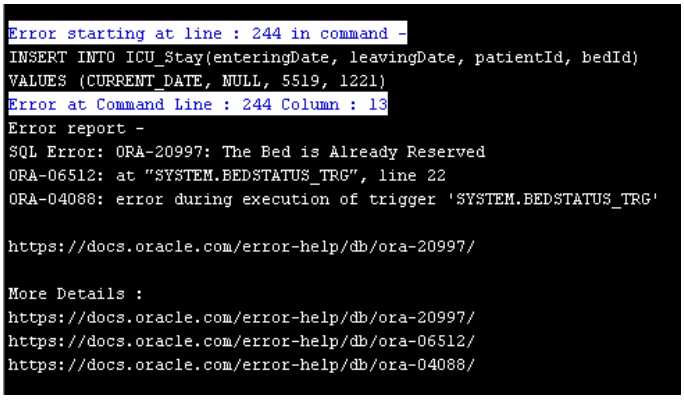


Figure 17: Result of Inserting a Row in ICU_Stay Table

Test Update

```
UPDATE ICU_Stay
SET leavingDate = CURRENT_DATE
WHERE stayId = 10237;
```

I tried to update an ICU stay and end that stay to see its effect on the ICU_bed table.

```
SELECT s.stayId, s.leavingDate, b.bedId, b.status
from icu_stay s
INNER JOIN ICU_bed b
ON s.bedId = b.bedId
Where s.stayId = 10237;
```

STAYID	LEAVINGDATE	BEDID	STATUS
10237	31-AUG-26	1221	Available

Figure 18: Effect of Ending a Stay on Bed Status

The bed with bedId = 1221 before ending the stay had a status = In use but after ending the stay the bed status changed to Available.

Test Delete

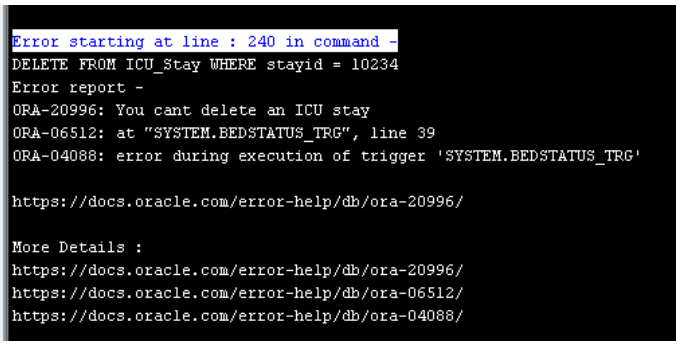


Figure 19: Result of Deleting a Row in ICU_Stay Table

2.2.2 Second Trigger

I created a row level trigger that checks the hospital transfer before any insert or update on Hospital_Transfer table, as the trigger retrieves the bed, entering date, leaving date of the selected ICU stay, and it retrieves the hospital of that bed, after that, it checks different things such as:

- The source hospital, as it should match the ICU stay hospital.
- The transfer date, as it should not be before the ICU stay entering date.
- Check if the ICU stay is still active when inserting (creating a transfer).
- Ending the ICU stay at the source hospital when a transfer is created.

If any of these conditions are not valid then the trigger will raise an application error and prevent the transfer.

Trigger PL/SQL Code

```
CREATE OR REPLACE TRIGGER checkTransfers_TRG
BEFORE INSERT OR UPDATE ON Hospital_Transfer
FOR EACH ROW

DECLARE
bedIdVar INT;
enteringDateVar DATE;
leavingDateVar DATE;
hospitalIdVar INT;

BEGIN

SELECT bedId, enteringDate, leavingDate
INTO bedIdVar, enteringDateVar, leavingDateVar
FROM ICU_Stay
WHERE stayId = :NEW.stayId;

SELECT hospitalId
INTO hospitalIdVar
FROM ICU_BED
WHERE bedId = bedIdVar;

IF :NEW.fromhospitalId != hospitalIdVar THEN
RAISE_APPLICATION_ERROR(-20994, 'Source Hospital Does Not Match the ICU Stay
Hospital');
END IF;

IF :NEW.transferDate < enteringDateVar THEN
RAISE_APPLICATION_ERROR(-20993, 'Transfer Date Cant be Before ICU Stay Entering
Date');
END IF;

IF leavingDateVar IS NOT NULL THEN
RAISE_APPLICATION_ERROR(-20992, 'Transfer Cannot Be Completed, The ICU Stay is
Already Finished');
END IF;


IF INSERTING AND leavingDateVar IS NULL THEN
UPDATE ICU_Stay
SET leavingDate = :NEW.transferDate
WHERE stayId = :NEW.stayId;
END IF;

END checkTransfers_TRG;
```

Test Insert

I inserted into Hospital_Transfer table to create a transfer for an ICU stay that is not finished yet to see if the trigger automatically ends the stay when a transfer happens.

STAYID	ENTERINGDATE	LEAVINGDATE	BEDID	HOSPITALID
10429	18-AUG-26	(null)	1461	487

Figure 20: The Row Before Transferring

```
INSERT INTO Hospital_Transfer(transferDate, stayId, fromhospitalId, tohospitalId)
VALUES( CURRENT_DATE, 10429, 487, 488);
```

STAYID	ENTERINGDATE	LEAVINGDATE	BEDID	HOSPITALID
10429	18-AUG-26	01-SEP-26	1461	487

Figure 21: The Row After Transferring

As we can see in figure 21, after creating a transfer for that specific record, the stay has ended and leaving date was set to today's date.

Test Invalid Data

I inserted into Hospital_Transfer table to create a transfer, but the selected source hospital will not match the hospital where the ICU stay occurred to see if the trigger detects invalid data.

STAYID	ENTERINGDATE	LEAVINGDATE	BEDID	HOSPITALID
10430	17-AUG-26	(null)	1462	487

Figure 22: The Targeted Row

```
Error starting at line : 293 in command -
INSERT INTO Hospital_Transfer(transferDate, stayId, fromhospitalId, tohospitalId)
VALUES( CURRENT_DATE, 10430, 488, 489)
Error at Command Line : 293 Column : 13
Error report -
SQL Error: ORA-20994: Source Hospital Does Not Match the ICU Stay Hospital
ORA-06512: at "SYSTEM.CHECKTRANSFERS_TRG", line 20
ORA-04088: error during execution of trigger 'SYSTEM.CHECKTRANSFERS_TRG'
```

Figure 23: Result

As we can see in figure 23, the trigger prevented the transfer from being created.

2.2.3 Third Trigger

I created a row level trigger that manages the equipment status after an update on the ICU_Stay table, as when an ICU stay is ended, the trigger retrieve all equipment assigned to that ICU stay, then it changes the status of all equipment associated with that specific stay to 'Available'.

Trigger PL/SQL Code

```
CREATE OR REPLACE TRIGGER freeEquipment_TRQ
```

```
AFTER UPDATE ON ICU_Stay
FOR EACH ROW

DECLARE
CURSOR c1 IS
SELECT equipmentId
FROM Equipment_Stay
WHERE stayId = :NEW.stayId;

BEGIN

IF :OLD.leavingDate IS NULL AND :NEW.leavingDate IS NOT NULL THEN

FOR equipments IN c1 LOOP

UPDATE Equipment
SET status = 'Available'
WHERE equipmentId = equipments.equipmentId;

END LOOP;
END IF;

END freeEquipment_TRQ;
```

Test Update

I updated and end an active ICU stay that has equipment assigned to it to see if the trigger changes the equipment status to available.

STAYID	LEAVINGDATE	EQUIPMENTID	EQUIPMENTNAME	STATUS
10301	(null)	1751	Ventilator-01	In use

Figure 24: Record Before Updating

```
UPDATE ICU_Stay
SET leavingDate = CURRENT_DATE
WHERE stayId = 10301;
```

EQUIPMENTID	EQUIPMENTNAME	STATUS
1751	Ventilator-01	Available

Figure 25: Result After Updating

As we can see in figure 25, the status of the equipment is updated and is set to 'Available'.

2.2.4 Fourth Trigger

I created a row level trigger that manages the equipment status only when equipment is assigned to an ICU stay, as the trigger check if that ICU stay is still active, the equipment belongs to the same hospital where the ICU stay is, and if equipment is already reserved.

If stay is active, equipment belongs to the same hospital, and equipment is not reserved, the equipment will be reserved to a stay and the trigger changes the status of the equipment to “In use”.

Trigger PL/SQL Code

```
CREATE OR REPLACE TRIGGER equipmentStatus_TRG
BEFORE INSERT ON Equipment_Stay
FOR EACH ROW

DECLARE

statusVar VARCHAR(20);
equipmentHospitalIdVar INT;
stayHospitalIdVar INT;
bedIdVar INT;
leavingDateVar DATE;

BEGIN

SELECT status, hospitalId
INTO statusVar, equipmentHospitalIdVar
FROM Equipment
WHERE equipmentId = :NEW.equipmentId;

SELECT bedId, leavingDate
INTO bedIdVar, leavingDateVar
FROM ICU_Stay
WHERE stayId = :NEW.stayId;

SELECT hospitalId
INTO stayHospitalIdVar
FROM ICU_Bed
WHERE bedId = bedIdVar;

IF leavingDateVar IS NOT NULL THEN
RAISE_APPLICATION_ERROR(-20991, 'ICU Stay Has Already Finished');
END IF;

IF equipmentHospitalIdVar != stayHospitalIdVar THEN
RAISE_APPLICATION_ERROR(-20990, 'Equipment Does Not Belong to The Stay Hospital');
END IF;

IF statusVar = 'In use' Then
RAISE_APPLICATION_ERROR(-20989, 'The Equipment is Already Reserved');

ELSE

UPDATE Equipment
SET status = 'In use'
WHERE equipmentId = :NEW.equipmentId;

END IF;

END equipmentStatus_TRG;
```

Test Invalid Data

I inserted into Equipment_Stay table to reserve an equipment, but the selected equipment will have a status = “In use” to see if the trigger detects invalid data.

STAYID	LEAVINGDATE	HOSPITALID	EQUIPMENTID	EQUIPMENTNAME	STATUS
10302	(null)	471	1752	Ventilator-02	In use

Figure 26: Targeted Row

```
Error starting at line : 375 in command -
INSERT INTO Equipment_Stay(stayId, equipmentId)
VALUES(10302, 1752)
Error at Command Line : 375 Column : 13
Error report -
SQL Error: ORA-20989: The Equipment is Already Reserved
ORA-06512: at "SYSTEM.EQUIPMENTSTATUS_TRG", line 35
ORA-04088: error during execution of trigger 'SYSTEM.EQUIPMENTSTATUS_TRG'
```

Figure 27: Result

Test Insert

I inserted onto the Equipment_Stay table to reserve an available equipment for an active ICU stay. After the insert we will see if the trigger changes the equipment status from Available to In use.

EQUIPMENTID	EQUIPMENTNAME	STATUS
1744	InfusionPump-01	Available

Figure 28: Targeted Row

```
INSERT INTO Equipment_Stay(stayId, equipmentId)
VALUES(10293, 1744);
```

EQUIPMENTID	EQUIPMENTNAME	STATUS
1744	InfusionPump-01	In use

Figure 29: Result After Inserting

2.2.5 Fifth Trigger

The last trigger I created is a statement level trigger, and since daily charge contains billing records that must not be changed or deleted, the trigger protect these bills from being updated or deleted.

Trigger PL/SQL Code

```
CREATE OR REPLACE TRIGGER protectBills_TRG
BEFORE UPDATE OR DELETE ON Daily_Charge
BEGIN
RAISE_APPLICATION_ERROR(-20988, 'Bills Records Cannot Be Deleted Or Updated');
```

```
END protectBills_TRG;
```

Test Delete/Update

```
Error starting at line : 374 in command -
UPDATE Daily_Charge
SET amount = 500
WHERE chargeId = 34476
Error at Command Line : 374 Column : 8
Error report -
SQL Error: ORA-20988: Bills Records Cannot Be Deleted Or Updated
ORA-06512: at "SYSTEM.PROTECTBILLS_TRG", line 2
ORA-04088: error during execution of trigger 'SYSTEM.PROTECTBILLS_TRG'
```

Figure 30: Result of Updating

```
Error starting at line : 374 in command -
DELETE FROM Daily_Charge
WHERE chargeId = 34476
Error at Command Line : 374 Column : 13
Error report -
SQL Error: ORA-20988: Bills Records Cannot Be Deleted Or Updated
ORA-06512: at "SYSTEM.PROTECTBILLS_TRG", line 2
ORA-04088: error during execution of trigger 'SYSTEM.PROTECTBILLS_TRG'
```

Figure 31: Result of Deleting

2.3 PL/SQL Package

I created a package that groups related billing subprograms together, as addBill_PRC is a procedure that adds a new bill to an ICU stay and record the transaction in the log table, the getStayTotal_FUNC that was already implemented was added to the package because it is related to billing, the getBillCount_FUNC is a function that return the number of bills for an ICU stay, and the addLog_PRC is used inside the package to insert the transaction details to the log table.

2.3.1 Audit Log Table

```
CREATE TABLE log(
logId INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
actionType VARCHAR(50) NOT NULL,
actionDate DATE DEFAULT CURRENT_DATE NOT NULL,
details VARCHAR(100),
stayId INT NOT NULL,
CONSTRAINT "ICU_Stay-Log_FK" FOREIGN KEY(stayId)
REFERENCES ICU_Stay(stayId)
);
```

2.3.2 Package PL/SQL Code

Specification:

```
CREATE OR REPLACE PACKAGE billing_PKG AS
```

```
PROCEDURE addBill_PRC(
stayIdParameter IN INT,
chargeDateParameter IN DATE,
amountParameter IN DECIMAL
);

FUNCTION getStayTotal_FUNC(
stayIdParameter IN INT
)
RETURN DECIMAL;

FUNCTION getBillCount_FUNC(
stayIdParameter IN INT
)
RETURN INT;

END billing_PKG;
```

Body:

```
CREATE OR REPLACE PACKAGE BODY billing_PKG AS

--prc 1
PROCEDURE addLog_PRC(
stayIdParameter IN INT,
actionTypeParameter IN VARCHAR,
detailsParameter IN VARCHAR
) IS
BEGIN

INSERT INTO log(actionType, details, stayId)
VALUES(actionTypeParameter, detailsParameter, stayIdParameter);

END addLog_PRC;

--prc 2
PROCEDURE addBill_PRC(
stayIdParameter IN INT,
chargeDateParameter IN DATE,
amountParameter IN DECIMAL
) IS

enteringDateVar DATE;
leavingDateVar DATE;

BEGIN

SELECT enteringDate, leavingDate
INTO enteringDateVar, leavingDateVar
FROM ICU_Stay
WHERE stayId = stayIdParameter;

IF chargeDateParameter < enteringDateVar THEN
RAISE_APPLICATION_ERROR(-20987, 'Bill Date Cannot Be Before The Stay Entering Date');
END IF;

IF leavingDateVar IS NOT NULL THEN
RAISE_APPLICATION_ERROR(-20986, 'Cannot Add Bill To an Ended Stay');
```

```
END IF;

INSERT INTO Daily_Charge(chargeDate, amount, stayId)
VALUES(chargeDateParameter, amountParameter, stayIdParameter);

addLog_PRC(stayIdParameter, 'Bill Added', 'Bill Amount = ' || amountParameter);
DBMS_OUTPUT.PUT_LINE('Bill Was Successfully Added For Stay With Id = ' ||
stayIdParameter);

COMMIT;

EXCEPTION
WHEN NO_DATA_FOUND THEN
RAISE_APPLICATION_ERROR(-20985, 'ICU Stay Does Not Exist');

END addBill_PRC;

--func 1
FUNCTION getStayTotal_FUNC(
stayIdParameter IN INT
)
RETURN DECIMAL IS

totalChargesPerStay DECIMAL(10,2);

BEGIN

SELECT SUM(amount)
INTO totalChargesPerStay
FROM Daily_Charge
WHERE stayId = stayIdParameter;

IF totalChargesPerStay IS NULL THEN
RAISE_APPLICATION_ERROR(-20984, 'The Stay Has No Charges Yet');
ELSE
RETURN totalChargesPerStay;
END IF;

END getStayTotal_FUNC;

--func 2
FUNCTION getBillCount_FUNC(
stayIdParameter IN INT
)
RETURN INT IS

billCountVar INT;
stayIdVar INT;

BEGIN

SELECT stayId
INTO stayIdVar
FROM ICU_Stay
WHERE stayId = stayIdParameter;

SELECT COUNT(*)
INTO billCountVar
FROM Daily_Charge
WHERE stayId = stayIdParameter;
```

```
RETURN billCountVar;

EXCEPTION
WHEN NO_DATA_FOUND THEN
RAISE_APPLICATION_ERROR(-20983, 'ICU Stay Does Not Exist');

END getBillCount_FUNC;

END billing_PKG;
```

2.3.3 Testing

Testing The Log

I inserted a new bill to an active stay using the addBill procedure and then I checked the log table to see if the transaction was recorder successfully.

```
BEGIN
billing_PKG.addbill_prc(10238, CURRENT_DATE, 876);
END;
```

LOGID	ACTIONTYPE	ACTIONDATE	DETAILS	STAYID
1	Bill Added	01-SEP-26	Bill Amount = 876	10238

Figure 32: Impact on Log Table

As we can see, after adding the bill, it was successfully recorded to the log table.

Testing Bill Count Function

I tested the getBillCount function to see if the function returns the number of bills for a specific stay.

```
DECLARE
billCount INT;
BEGIN
billCount := billing_PKG.getBillCount_FUNC(10238);
DBMS_OUTPUT.PUT_LINE('Bills: ' || billCount);
END;
```

```
Bills: 3


PL/SQL procedure successfully completed.
```

Figure 33: Result

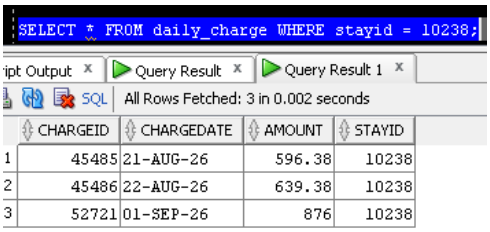


Figure 34: Bills from Daily_Charge Table

As we can see it returned the number of bills associated with the stay that has the id = 10238.

2.4 Evaluation

In my project, I implemented a very big part of the business logic directly inside Oracle including implementing procedures, functions, triggers, and a package, and running business logic inside Oracle or inside the application have strengths and weaknesses.

On one hand, running business logic inside Oracle means the business logic is executed close to the data which reduces the communication between the application and the database because logic and data are in the same location, for example, in my project, when an ICU stay is created, there is a trigger called bedStatus_TRG that checks the selected bed status and changes its status directly inside oracle which means all the processing is directly inside Oracle, while if this logic was inside the application, then the application must first retrieve the bed status from oracle, check if the bed is available, and then send an update statement of the bed status to Oracle, which means that there will be more communication between the application and the database server. Reducing the communication between the application and the database server reduces the number of requests sent between them which reduces network traffic and makes the response time faster.

Another thing is that running business rules inside Oracle means that the rules are enforced directly on the database, as if these rules were only implemented inside the application, then they would work only inside the application, and if another application or used accesses Oracle directly, those business rules are not enforced because they are only enforced in the application. Keeping these critical rules inside Oracle makes them apply whenever something happens regardless of the application or the user who executes the operation, that's why I applied most of the critical logic inside Oracle itself.

But, running a lot of business logic inside Oracle can have some scalability issues because the database server is not only responsible for storing and retrieving data but also responsible for executing procedures, functions, triggers, and packages, so when the number of users and transactions increases, the amount of processing on the same database server will also increase, for example, imagine we are inserting or updating a

large number of patients, stays, transfers, or equipment assignment and we have a row level triggers that will execute because of an action performed to these tables, as it will execute for every affected row and slow the database.

On the other hand, running business logic inside the application can reduce some of the processing on the database server because some of the processing like checks, calculations, and decisions will be done by the application instead, and the oracle only receives the final operations like INSERT or UPDATE, so the logic implemented inside the application does not fully remove the workload on the database but it only moves some part of the processing away from the database server.

Also, when the number of the users increases, the application can handle different processing by running on more than one server, for example, 1000 users can be divided between several application servers, each server process requests for part of the users, but if the business logic is only inside Oracle, then all of these users will still use the same database server to execute the logic which can put most of the processing load on the database server, making it slower.

But running business logic inside the application has some issues because as I mentioned before, some operations may require more communication between the application and the database server, and some rules may be skipped if another application or user accesses the database directly. Also, one business operation may require several statements and if the application loses the connection with the database server before completing the statement, part of the operation will not be completed, for example, transferring a patient need to end the stay, release the bed, and release the equipment, and if the application loses the connection after only ending the stay, then bed and equipment is not released which makes the information inaccurate.

So, running business logic inside Oracle and running business logic inside the application have strengths and weaknesses, so I don’t think all the business logic should be placed in one place. Most critical rules that directly protect the database such as checking availability, validating hospital transfers, and preventing bills from being updated or deleted should stay inside Oracle because they should always be enforces regardless of which application or user access the database, but the logic that is related to the application workflow and does not directly protect the data can be executed inside the application such as suggesting the hospitals with available beds when a patient need to be transferred which reduces some of the processing on the database server and allow the application processing to scale separately.

Part 3: Multi-Model NoSQL Implementation

3.1 Document Database

The Oracle database is used to manage the structured data of the ICU Beds & Hospital Capacity Management system like ICU stay, ICU beds, equipment allocations, etc. But there are some data related to an ICU stay may not always have the same structure, so, MongoDB will be used to store semi-structured medical information related to ICU stays such as clinical notes, monitoring logs, and incident notes.

I exported the ICU stay records from the Oracle database as JSON files to use stayId values in MongoDB so that we can link each MongoDB document with its related ICU stay in the Oracle database. After exporting the ICU stays from Oracle, I will add this semi-structured medical information to the MongoDB documents.

3.1.1 How Did I Export Data From Oracle

First, I used a query to retrieve stayId of all the records in the Oracle database.

```
SELECT stayId
FROM ICU_Stay
ORDER BY enteringDate;
```

Then, I clicked on the export option as shown in figure 35, then I exported the data as JSON as shown in figure 36.

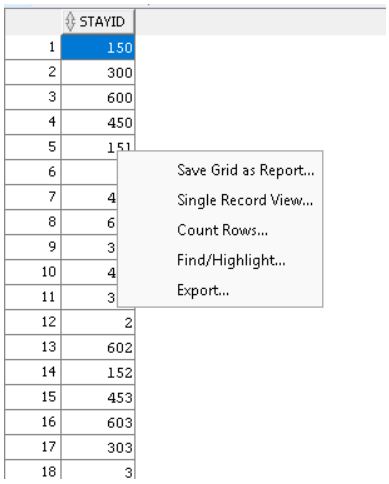


Figure 35: Export Option

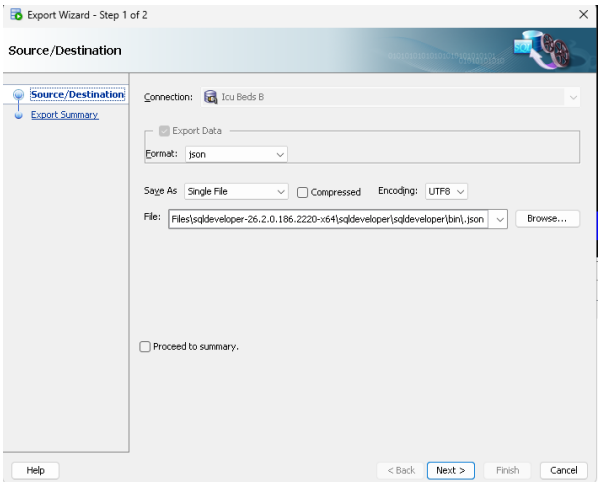


Figure 36: Exporting Data

3.1.2 Collection

I created a collection called Stay_Info to store the semi-structured medical information related to ICU stays. Each document has a mandatory field called stayId and optional fields called clinicalNotes, monitoringLogs, and incidentNotes and they may not appear in every document because information can differ from one ICU stay to another.

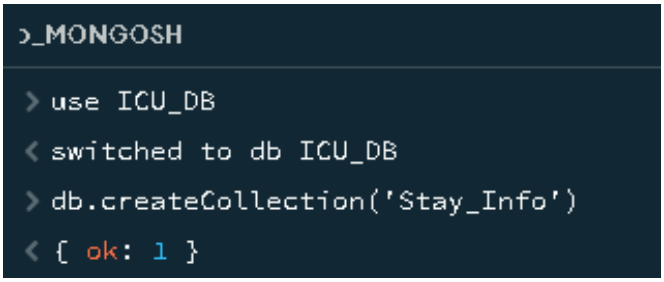


Figure 37: Creating the Collection

3.1.3 Adding Documents

After exporting the ICU stay records from Oracle, I used the stayId in each MongoDB document and added the semi-structured medical information to the documents such as clinicalNotes, monitoringLogs, and incidentNotes. Since there are 12000 records for ICU stays, I used chatgpt to generate the data to be stored in clinicalNotes, monitoringLogs, and incidentNotes.

Small Sample of the Inserted Documents

```
db.Stay_Info.insertMany([
{
stayId:10238,
clinicalNotes:["Patient remained under close observation during the night shift."],
monitoringLogs:[
{
time:"08:00",
```

```
temperature:38.2,
heartRate:105
},
{
time:"12:00",
oxygenSaturation:96
}]},
{
stayId:10301,
monitoringLogs:
[{
time:"10:00",
oxygenSaturation:91
}],
incidentNotes:["Oxygen saturation dropped during the morning and additional
monitoring was required."]},
{
stayId:10429,
clinicalNotes:["Patient remained stable during the shift."]}
])
```

3.1.4 Find Operations

Count

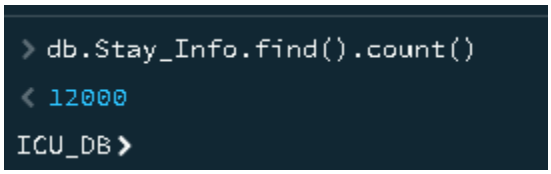


Figure 38: Count Result

Find a Specific Stay



Figure 39: Find Specific Stay Result

Find Stays in Order

```
> db.Stay_Info.find().limit(2).sort({stayId: 1})
< {
    _id: ObjectId('6a9882d46133bf68f212f801'),
    stayId: 1,
    monitoringLogs: [
      {
        time: '04:00',
        temperature: 38.2,
        heartRate: 114,
        oxygenSaturation: 97,
        bloodPressure: '142/75'
      },
      {
        time: '08:00',
        bloodPressure: '112/93'
      },
      {
        time: '12:00',
        heartRate: 120,
        oxygenSaturation: 89,
        bloodPressure: '139/76'
      },
      {
        time: '20:00',
        temperature: 37.7,
        heartRate: 73,
        oxygenSaturation: 90,
        bloodPressure: '135/90'
      }
    ],
    incidentNotes: [
      'A sudden change in breathing pattern was observed and documented.'
    ]
  }
```

```
{
  _id: ObjectId('6a9882d46133bf68f212f807'),
  stayId: 2,
  monitoringLogs: [
    {
      time: '00:00',
      temperature: 37.4,
      bloodPressure: '101/67'
    },
    {
      time: '04:00',
      heartRate: 78,
      oxygenSaturation: 97,
      bloodPressure: '151/65'
    }
  ]
}
```

Figure 40: Find and Sort Result

3.2 Graph Database

Before designing the graph database, I asked myself, what question should the graph database answers, and the main question I wanted the graph database to answer is which hospitals are connected to a selected hospital through incoming and outgoing transfers and what is the shortest route path between hospitals?

So, Neo4j was used to model two types of connections between hospitals, the first is based on actual inter-hospital transfers that were stored in Oracle. Hospitals are represented as nodes and each transfer is a relationship between the hospital that the ICU stay was transferred from and the hospital that the ICU stay was transferred to. The relationship is called TRANSFERRED_STAY_TO and it stores transferId, stayId, and transferDate, and the stayId was included because each hospital transfer is for a specific ICU stay, so the relationship represent actual historical ICU stays and means that two hospitals have a transfer connection through a stay.

The second connection is used to represent routes between hospitals, and the relationship between hospitals is called ROUTE which will be used to calculate the shortest route path between hospitals based on the minimum number of connections. Hospitals in the same city will be directly connected to each other, and only some hospitals are connected to hospitals in other cities, and the shortest path here represent a point that the route passes through and does not mean that the patient must enter the hospital in the middle of the route.

3.2.1 How Did I Export Data from Oracle

First, I used a query to retrieve hospitalId and hospitalName from the Hospital table in Oracle.

```
SELECT hospitalId, hospitalName
FROM Hospital
ORDER BY hospitalId;
```

Then I used another query to retrieve all the data in Hospital_Transfer table.

```
SELECT * FROM Hospital_Transfer
ORDER BY transferId;
```

Then I also used another query to retrieve the city of each hospital to be used only for generating synthetic route data, and since there are a large number of routes, I used an AI tool to help generate the routes between hospitals. The city was exported to help create a realistic routes between hospitals and wasn't added as a property in Neo4j.

```
SELECT hospitalId, hospitalName, city
FROM Hospital
ORDER BY city, hospitalId;
```

Then, I clicked on the export option as shown in figure 41, then I exported the data as CSV as shown in figure 42.

	TRANS...	TRANSFERDATE	STAYID	FROMHOSPITALID	TOHOSPITALID
1	1	11-JAN-24	1	1	2
2	2	14-JAN-24	2	2	3
3	3	17-JAN-24	3	3	4
4	4	20-JAN-24	4	4	5
5	5	18-JAN-24	5		6
6	6	21-JAN-24	6		7
7	7	24-JAN-24	7		8
8	8	27-JAN-24	8		9
9	9	30-JAN-24	9		10
10	10	28-JAN-24	10		11
11	11	31-JAN-24	11	11	12

Save Grid as Report...
Single Record View...
Count Rows...
Find/Highlight...
Export...

Figure 41: Export Option

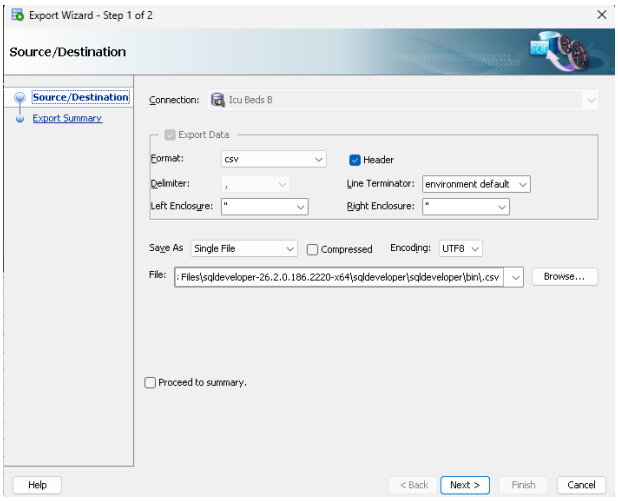


Figure 42: Exporting Data

3.2.2 Creating Hospital Nodes

After exporting the data from Oracle I loaded the data from the CSV file and created a node for each hospital using LOAD CSV WITH HEADERS.

```
1 LOAD CSV WITH HEADERS FROM 'file:///Hospitals.csv' AS row
2 MERGE (h:Hospital {hospitalId: row.HOSPITALID})
3 SET h.hospitalName = row.HOSPITALNAME;
```

Created 600 nodes, set 1,200 properties, added 600 labels

Figure 43: Creating the Nodes

After creating the nodes, I converted the hospitalId's from string to integers.

```
1 MATCH (h:Hospital)
2 SET h.hospitalId = toInteger(h.hospitalId);
```

Set 600 properties

Figure 44: Conversion

3.2.3 Creating Hospital Transfers Relationships

Then, I created the transfer relationship between the hospitals using the transfers CSV file as fromHospitalId is used to find the hospital that the stay was transferred from, and toHospitalId is used to find the hospital that the stay was transferred to.

```
LOAD CSV WITH HEADERS FROM 'file:///transfers.csv' AS row
MATCH (h1:Hospital {hospitalId: row.FROMHOSPITALID})
MATCH (h2:Hospital {hospitalId: row.TOHOSPITALID})
CREATE (h1)-[:TRANSFERRED_STAY_TO {transferId: row.TRANSFERID, stayId: row.STAYID, transferDate: row.TRANSFERDATE}]->(h2);
```

Created 700 relationships, set 2,100 properties

Figure 45: Creating Transfer Relationship

After creating the relationships, I converted some of the relationship properties from string to integers.

```
MATCH ()-[r:TRANSFERRED_STAY_TO]->()
SET r.transferId = toInteger(r.transferId), r.stayId = toInteger(r.stayId);
```

Set 1,400 properties

Figure 46: Conversion

3.2.4 Creating the Routes Between Hospitals

Then I created another file that contains the possible route connections and created the relationship between hospitals using the routes CSV file without including the city as a property or node. I converted the loaded hospitalIds from string to integer so they match the existing Hospital nodes.

```
LOAD CSV WITH HEADERS FROM 'file:///routes.csv' AS row
MATCH(h1:Hospital{hospitalId: toInteger(row.HOSPITAL1ID)})
MATCH(h2:Hospital{hospitalId: toInteger(row.HOSPITAL2ID)})
MERGE(h1)-[:ROUTE]->(h2);
```

Created 14,736 relationships

Figure 47: Creating Route Relationship

3.2.5 Cypher Queries

Hospital Transfer Graph

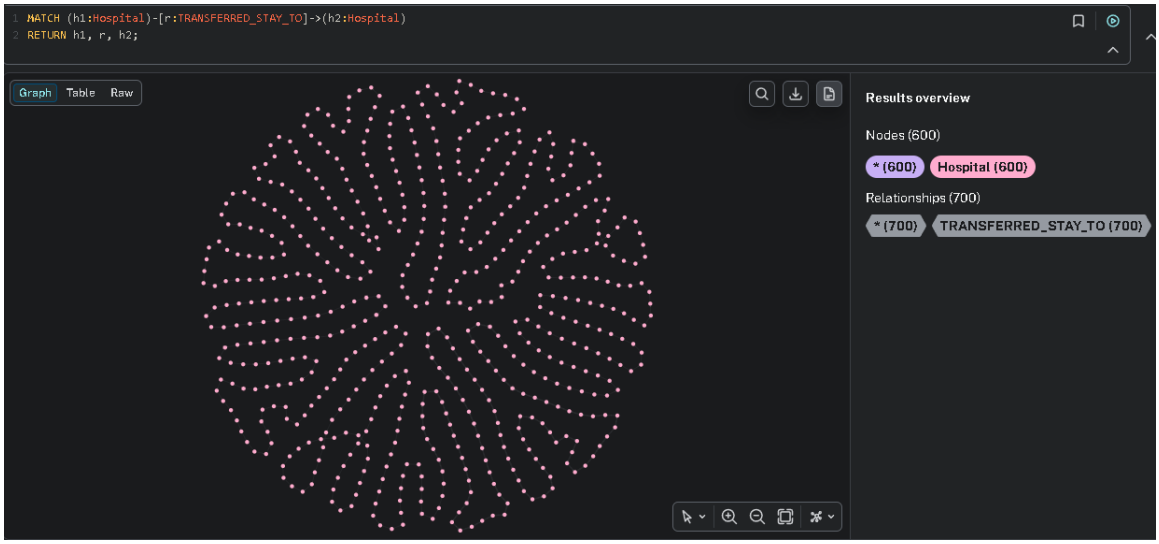


Figure 48: Graph

Hospital Routes Graph

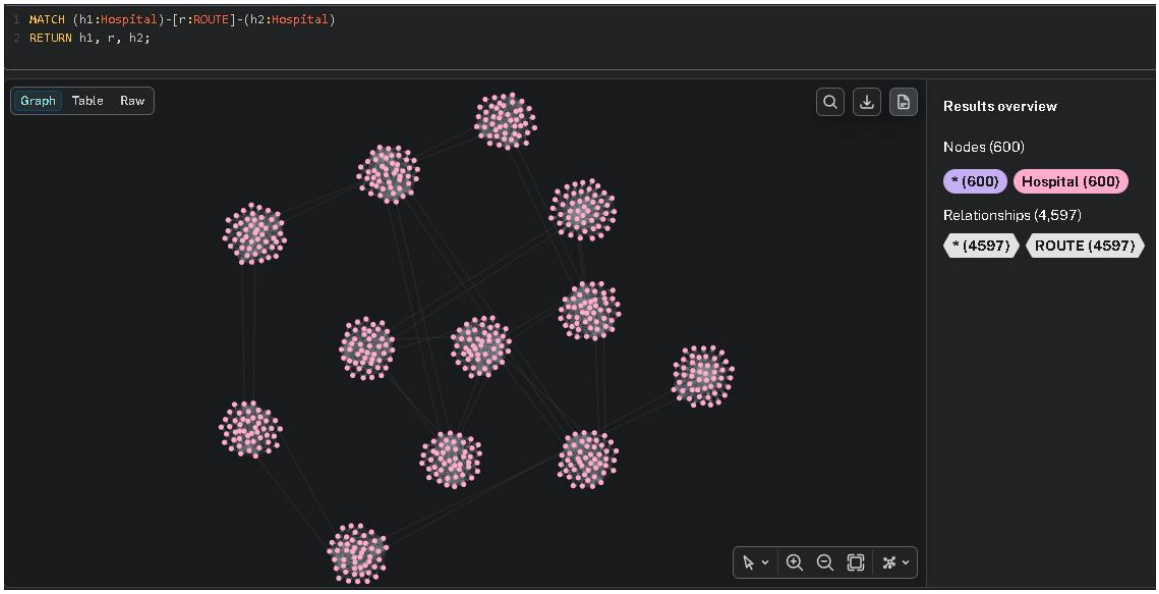


Figure 49: Graph

Finding the Shortest Path Between Two Hospitals

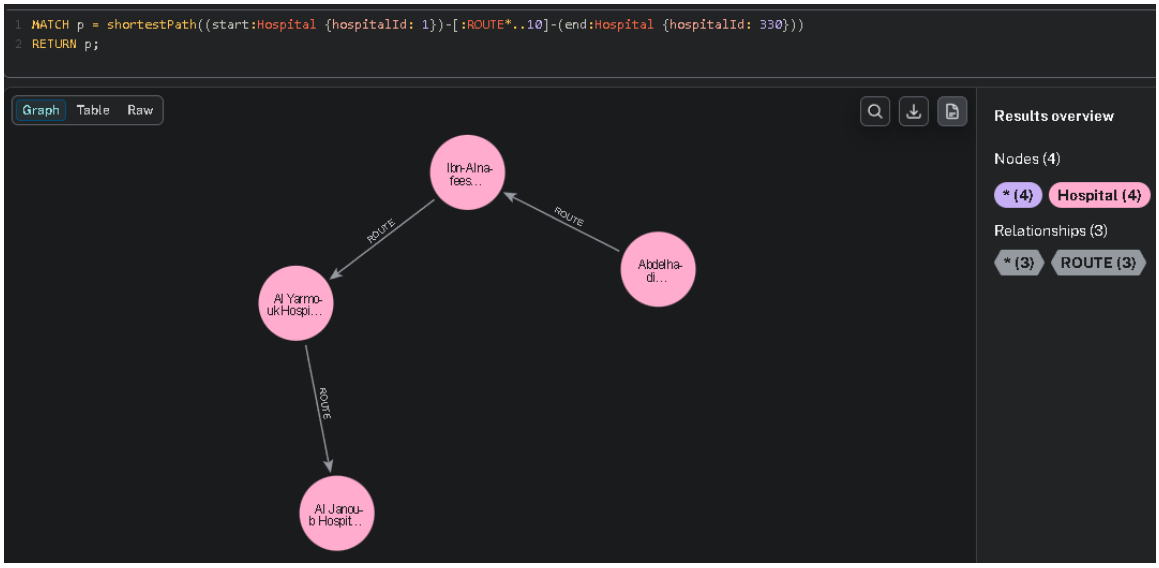


Figure 50: Query & Result

Finding Outgoing Transfers from a Specific Hospital

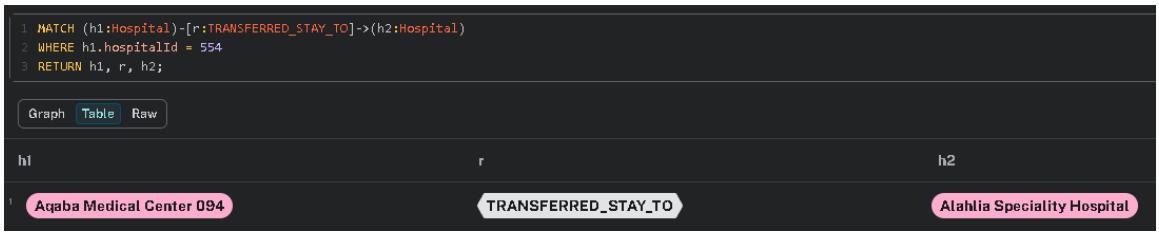


Figure 51: Query & Result

Finding Incoming Transfers from a Specific Hospital



Figure 52: Query & Result

Number of Transfers

```
MATCH (h1:Hospital)-[r:TRANSFERRED_STAY_TO]->(h2:Hospital)
RETURN count(r);
```

Table Raw

count(r)
700

Figure 53: Query & Result

Number of Routes

```
MATCH(h1:Hospital)-[r:ROUTE]->(h2:Hospital)
RETURN count(r);
```

Table Raw

count(r)
14736

Figure 54: Query & Result

Finding Transfers on a Specific Date

```
MATCH (h1:Hospital)-[r:TRANSFERRED_STAY_TO]->(h2:Hospital)
WHERE r.transferDate = "04-AUG-24"
RETURN h1, r, h2;
```

Graph Table Raw

	h1	r	h2
1	Queen Rania Pediatric Ho...	TRANSFERRED_STAY_TO	Rahbat Al-Wardieh Hospi...
2	West Emergency Hospital	TRANSFERRED_STAY_TO	East Emergency Hospital
3	New Al Shefa Day Surger...	TRANSFERRED_STAY_TO	Prime Hospital
4	Al-Bassel Cardiology Ho...	TRANSFERRED_STAY_TO	Aqaba Medical Center 094

Figure 55: Query & Result

Schema Visualization

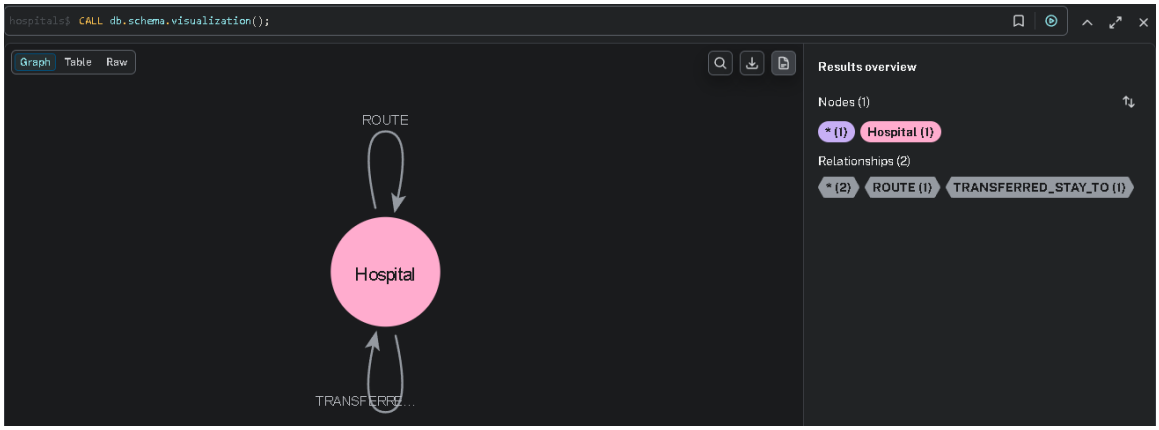


Figure 56: Query & Result

3.3 Performance Benchmarking

I searched for how to retrieve execution time of a query in Oracle, MongoDB, and Neo4j, and I will perform a query execution time tests using operations that are equivalent in Oracle, MongoDB, and Neo4j. Each query will be executed 5 times and calculate the average because the execution time may differ from one time to another, and the recorded execution time will be after we run the query multiple times.

All of the tests were done on the same test environment, locally, and on the same device.

3.3.1 MongoDB VS Oracle

For both MongoDB and Oracle, I will retrieve the stays where stayId is between 5000 and 10000 and then sort the result by stayId which allow the two databases to perform an equivalent operation on the same stay records.

MongoDB

For MongoDB I used new Date() to retrieve the time and also used toArray() because find() does not return all of the documents immediately but it returns a cursor which is used to access the results.

```
var before = new Date();
db.Stay_Info.find({stayId: {$gte: 5000, $lte: 10000}},{ _id: 0,  stayId:
1}).sort({stayId: 1}).toArray();
var after = new Date();
var execution_millis = after - before;
execution_millis;
```

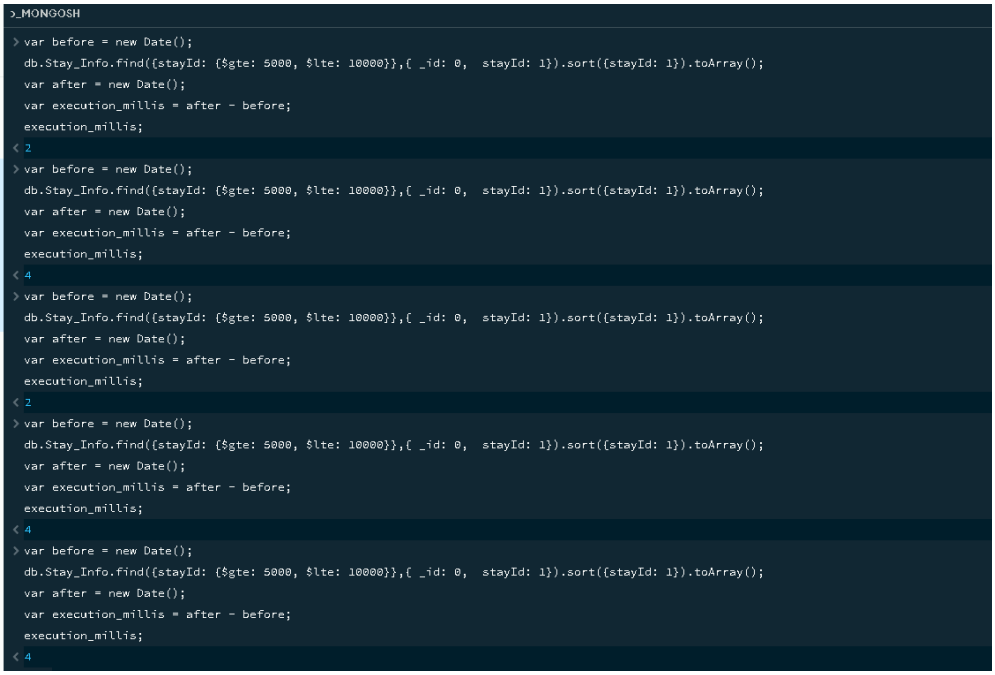


Figure 57: MongoDB Query Execution Time

As we can see from figure 57, we ran 5 tests on the query and the recorded execution times were different but resulted in an average of 3.2ms.

Oracle

For Oracle, I used SET TIMING ON and SET TIMING OFF which displays the time to execute the query.

```
SET TIMING ON;
SELECT stayId
FROM ICU_Stay
WHERE stayId >= 5000
AND stayId <= 10000
ORDER BY stayId ASC;
SET TIMING OFF;
```

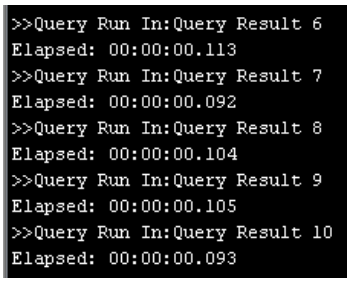


Figure 58: Oracle Query Execution Time

As we can see from figure 58, we ran 5 tests on the query and the recorded execution times were different but resulted in an average of 101.4ms.

Comparison

Database	Run 1	Run 2	Run 3	Run 4	Run 5	Average
MongoDB	2ms	4ms	2ms	4ms	4ms	3.2ms
Oracle	113ms	92ms	104ms	105ms	93ms	101.4 ms

As we can see MongoDB had an average execution time of 3.2 ms, and Oracle had an average execution time of 101.4 ms, so MongoDB performed faster for this filtering and sorting operation. However, this comparison was based on this specific query and does not mean that MongoDB will always perform faster than Oracle in all operations.

3.3.2 Neo4j VS Oracle

For both Neo4j and Oracle, I will retrieve all hospital transfers together with the source and destination hospital information, which allows which allow the two databases to perform an equivalent operation on the same hospital transfers records.

Neo4j

The hospital transfers information in Neo4j are stored in the relationship between the hospital nodes, so the query can retrieve these information from the relationship.

```
MATCH (h1:Hospital)-[r:TRANSFERRED_STAY_TO]->(h2:Hospital)
RETURN h1.hospitalId, h1.hospitalName, r.transferId, r.stayId, r.transferDate,
h2.hospitalId, h2.hospitalName;
```

For the execution time, I used "resultConsumedAfter" and "resultAvailableAfter" that are returned with the query and " resultAvailableAfter " represent the time until results become available and resultConsumedAfter " represent the additional time needed to consume the remaining results so I will add both values together as the complete time.



Figure 59: Neo4j Query Execution Times

As we can see from the figures, we ran 5 tests on the query and the recorded execution times were different but resulted in an average of 29.8ms.

Oracle

The hospital transfer information are stored in Hospital_Transfer table and the hospital information is stored in Hospital table so I used join to retrieve all of the required information. I also used the same SET TIMING method.

```
SET TIMING ON;
SELECT h1.hospitalId , h1.hospitalName , t.transferId, t.stayId, t.transferDate,
h2.hospitalId, h2.hospitalName
FROM Hospital_Transfer t
INNER JOIN Hospital h1
ON t.fromhospitalId = h1.hospitalId
INNER JOIN Hospital h2
ON t.tohospitalId = h2.hospitalId;
SET TIMING OFF;
```

```
>>Query Run In:Query Result 11
Elapsed: 00:00:00.098
>>Query Run In:Query Result 12
Elapsed: 00:00:00.089
>>Query Run In:Query Result 13
Elapsed: 00:00:00.100
>>Query Run In:Query Result 14
Elapsed: 00:00:00.088
>>Query Run In:Query Result 15
Elapsed: 00:00:00.101
```

Figure 60: Oracle Query Execution Time

As we can see from figure 60, we ran 5 tests on the query and the recorded execution times were different but resulted in an average of 95.2ms.

Comparison

Database	Run 1	Run 2	Run 3	Run 4	Run 5	Average
Neo4j	43ms	33ms	33ms	25ms	15ms	29.8ms
Oracle	98ms	89ms	100ms	88ms	101ms	95.2ms

As we can see, Neo4j had an average execution time of 29.8 ms, and Oracle had an average execution time of 95.2ms, so Neo4j performed faster for this test. However, this comparison was based on this specific query and does not mean that Neo4j will always perform faster than Oracle in all operations.

3.3.3 Overall Performance

The test strengthen the idea of using databases together instead of replacing SQL with NoSQL database, as SQL is important for the main structured data, MongoDB and Neo4j is important for other data, as the system can keep reliability and structure of the database while also using NoSQL for specific tasks where they can provide better performance.

Those test does not mean that NoSQL is always faster than Oracle, because the execution time can vary based on the query, the data, the operation, and the timing method, and the tests were performed locally on the same device.

The timing methods were also different. Oracle used SET TIMING, MongoDB used another time method, and Neo4j used resultConsumedAfter and resultAvailableAfter. So, the three execution times may not include exactly the same parts of the operation, as each query was tested five times, which gives an idea about the selected query, but it is not enough to decide the performance of the whole databases.

Part 4: Application Integration, Testing & System

4.1 Software Application

I created a website using PHP, HTML, CSS, and JavaScript and with the help of AI tools. The website connects to the three databases used in the project, as Oracle is used to manage the structured data of the ICU Beds & Hospital Capacity Management system, MongoDB is used to store the medical information related to ICU stays, and Neo4j is used to find the shortest route path between hospitals. The website allows Staff to select

their hospital and manage its ICU operations, while Admin can view the data of all hospitals.

4.1.1 Admin

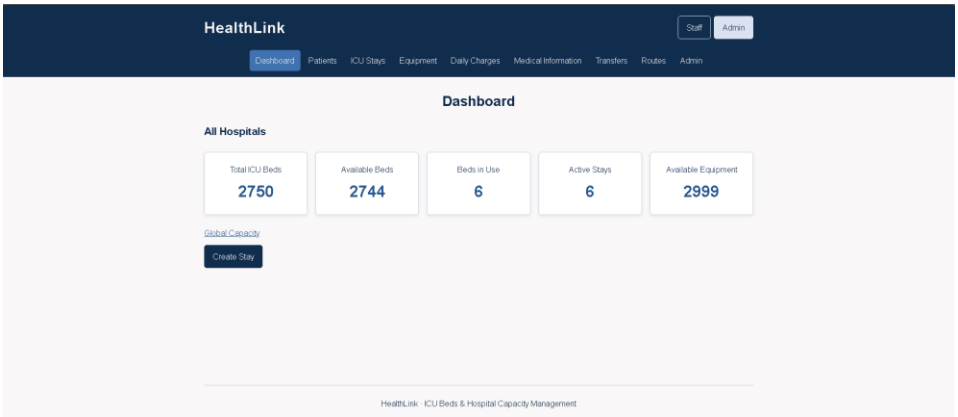


Figure 61: Admin Homepage

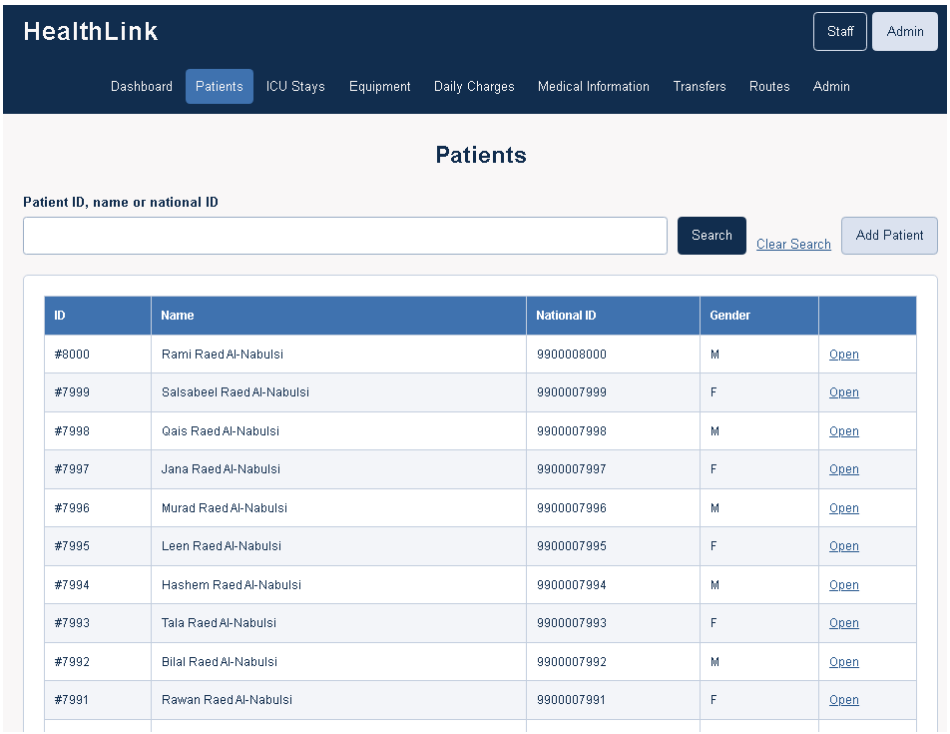


Figure 62: All Patients

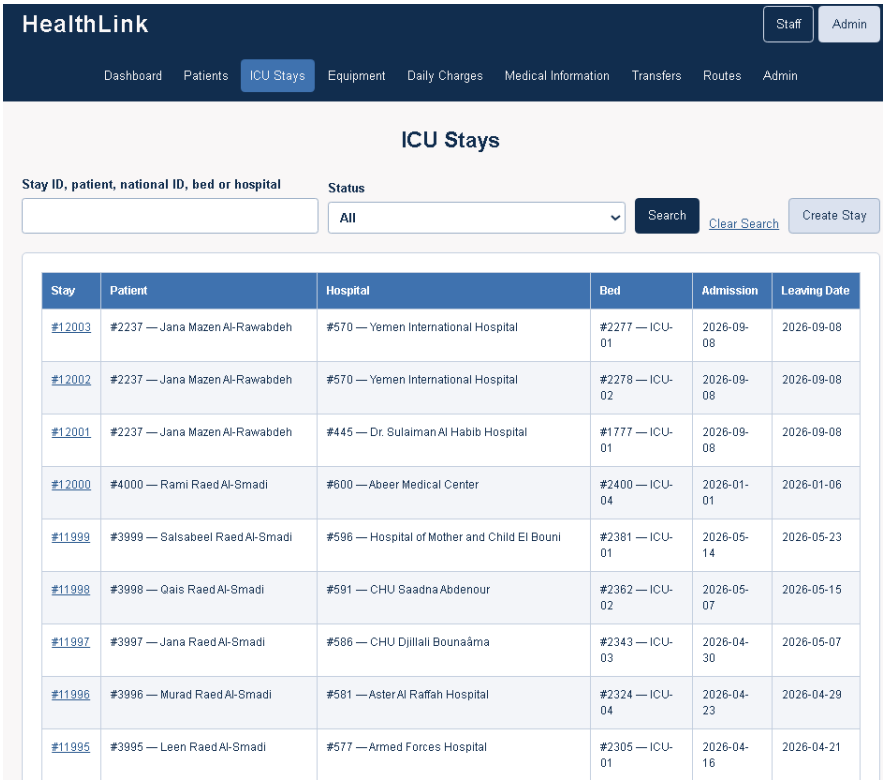


Figure 63: All ICU Stays

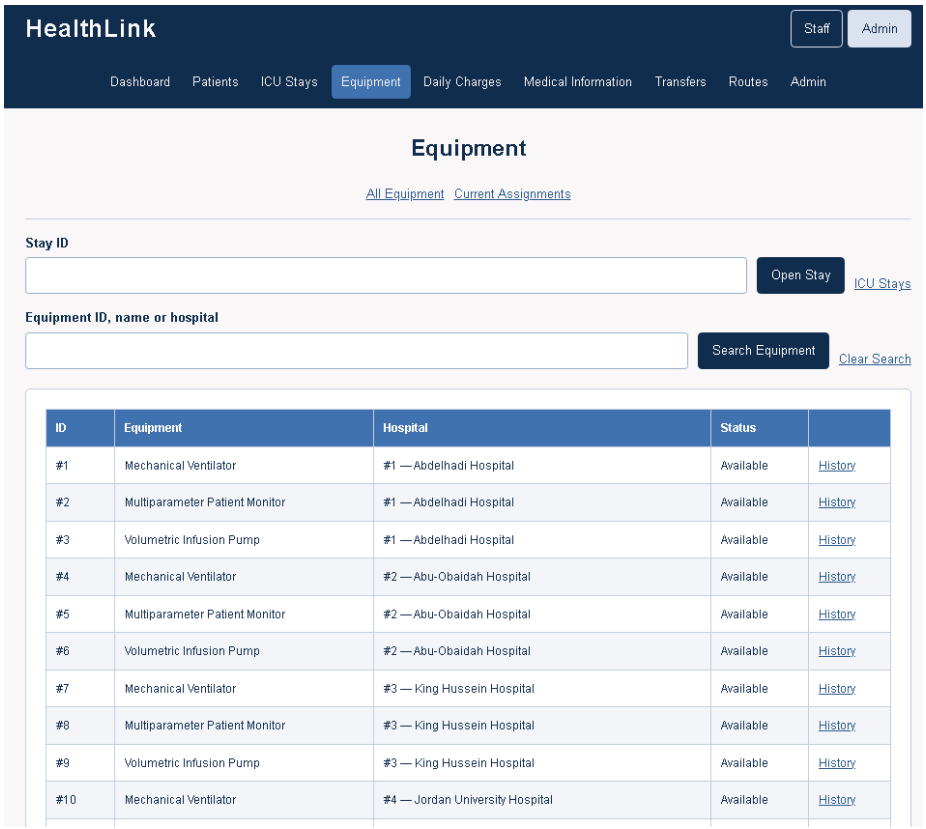


Figure 64: All Equipments

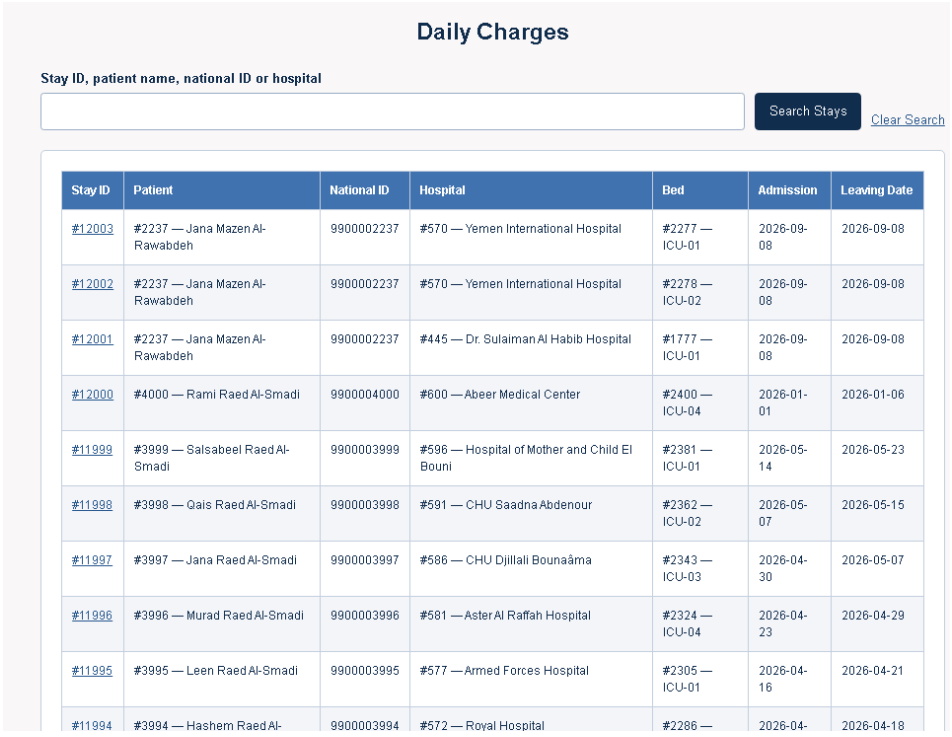


Figure 65: All Stays & Charges Per Stay

You have to press on the stayId to view the information.

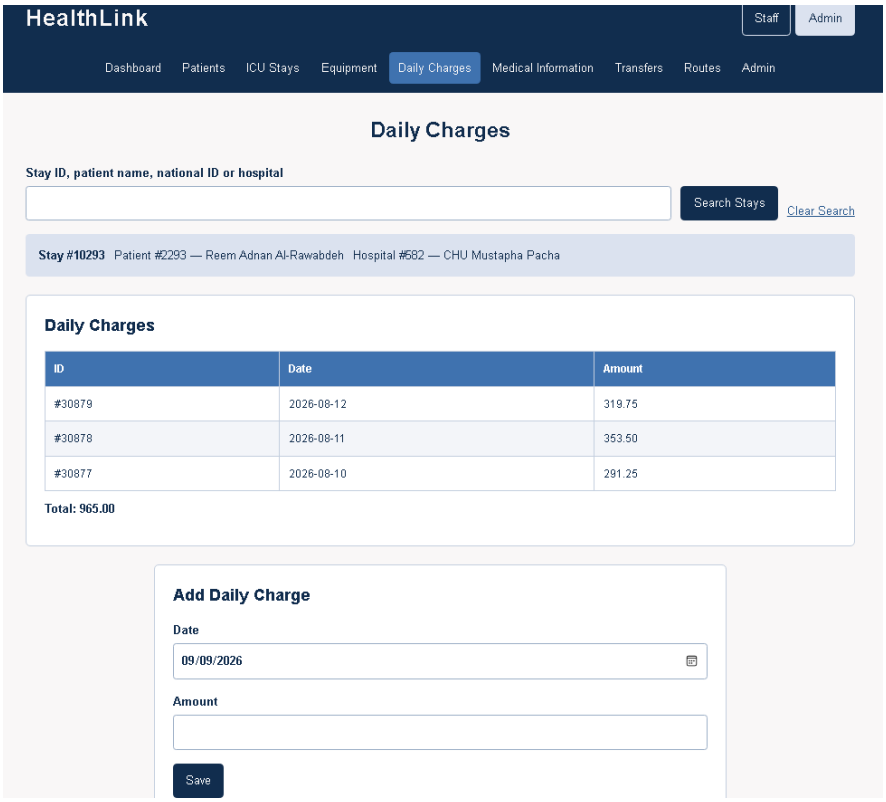


Figure 66: Charges Per Stay

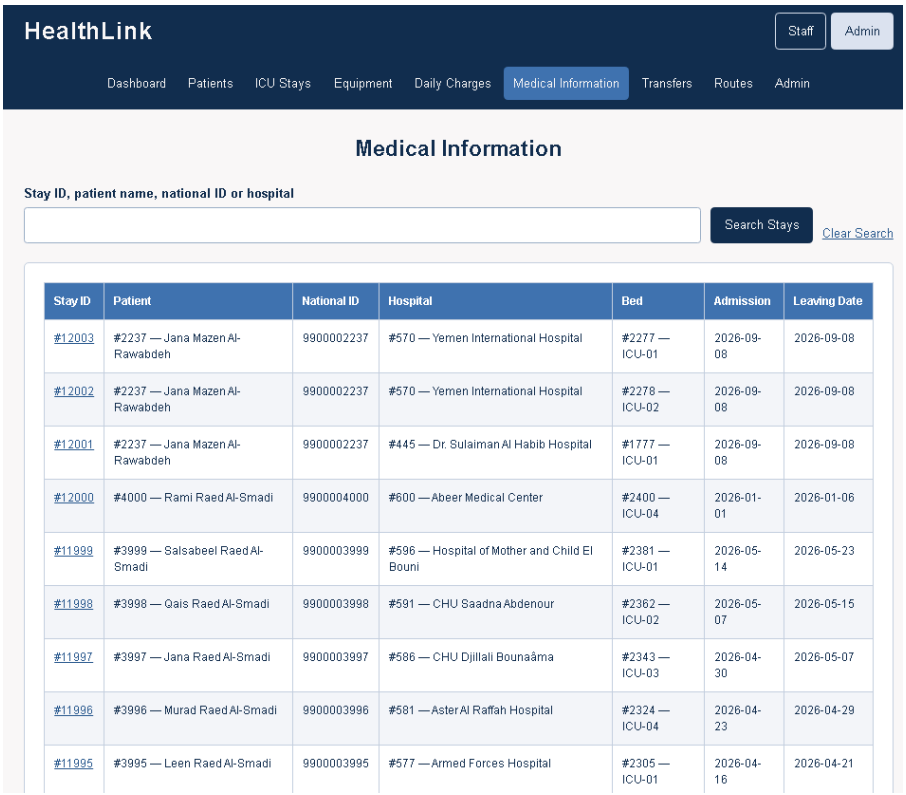


Figure 67: Stays & Medical Information Per Stay

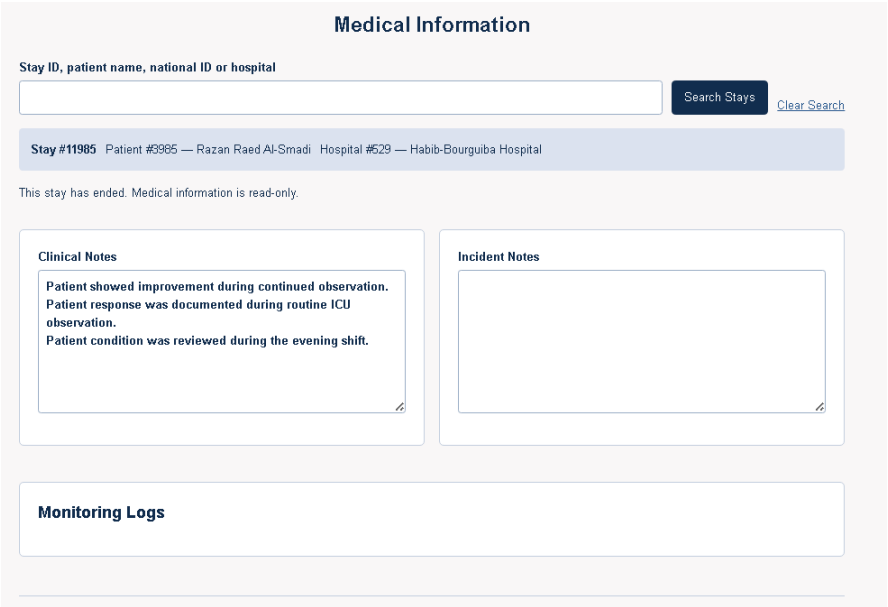


Figure 68: Medical Information Per Stay

Transfer History

Transfer ID, stay, patient or hospital

Search History | Clear Search

ID	Date	Stay	From	To	Patient
#700	2024-07-26	#700	#100 — Queen Alia Heart Institute	#101 — Queen Alia Military Hospital	#700 — Ibrahim Adnan Al-Majali
#699	2024-07-28	#699	#99 — Qasr Shabeeb Hospital	#100 — Queen Alia Heart Institute	#699 — Dima Adnan Al-Majali
#698	2024-07-25	#698	#98 — Princess Salma Hospital	#99 — Qasr Shabeeb Hospital	#698 — Ali Adnan Al-Majali
#697	2024-07-22	#697	#97 — Princess Raya Hospital	#98 — Princess Salma Hospital	#697 — Maya Adnan Al-Majali
#696	2024-07-19	#696	#96 — Princess Rahma Hospital	#97 — Princess Raya Hospital	#696 — Khaled Adnan Al-Majali
#695	2024-07-16	#695	#95 — Princess Iman Hospital	#96 — Princess Rahma Hospital	#695 — Aya Adnan Al-Majali
#694	2024-07-18	#694	#94 — Princess Haya Bint Al-Hussein Military Hospital	#95 — Princess Iman Hospital	#694 — Yousef Adnan Al-Majali
#693	2024-07-15	#693	#93 — Princess Basma Hospital	#94 — Princess Haya Bint Al-Hussein Military Hospital	#693 — Reem Adnan Al-Majali
#692	2024-07-12	#692	#92 — Prince Zaid Bin Al-Hussein Hospital	#93 — Princess Basma Hospital	#692 — Hamza Adnan Al-Majali
#691	2024-07-09	#691	#91 — Prince Rashid Bin Al-Hassan Hospital	#92 — Prince Zaid Bin Al-Hussein Hospital	#691 — Noor Adnan Al-Majali
#690	2024-07-06	#690	#90 — Prince Hussein Ben Abdulla II Hospital	#91 — Prince Rashid Bin Al-Hassan Hospital	#690 — Laith Adnan Al-Majali
#689	2024-07-08	#689	#89 — Prince Hussein Ben Abdulla II Center of Urology and Organ Transplant	#90 — Prince Hussein Ben Abdulla II Hospital	#689 — Hala Adnan Al-Majali

Figure 69: Transfer History

HealthLink

Staff | Admin

Dashboard | Patients | ICU Stays | Equipment | Daily Charges | Medical Information | Transfers | Routes | Admin

Routes

Source Hospital: #6 — Al Yarmouk Hospital

Destination Hospital: #151 — Al-Rashad Psychiatric Hospital

Find Route

Minimum-Hop Route

4 hops

1. Al Yarmouk Hospital #6
2. Ibn-Alnafees Hospital #61
3. Luzmila Hospital #73
4. Al Shaheed Abu Diah Hospital #31
5. Al-Rashad Psychiatric Hospital #151

Figure 70: Routes

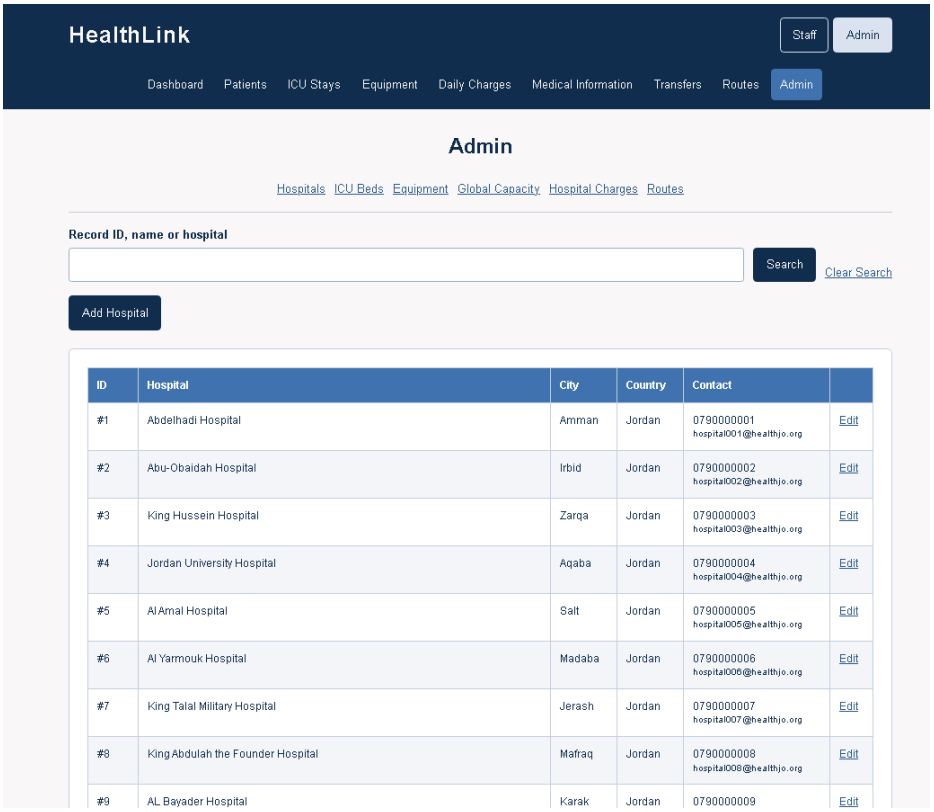


Figure 71: Actual Admin Operations

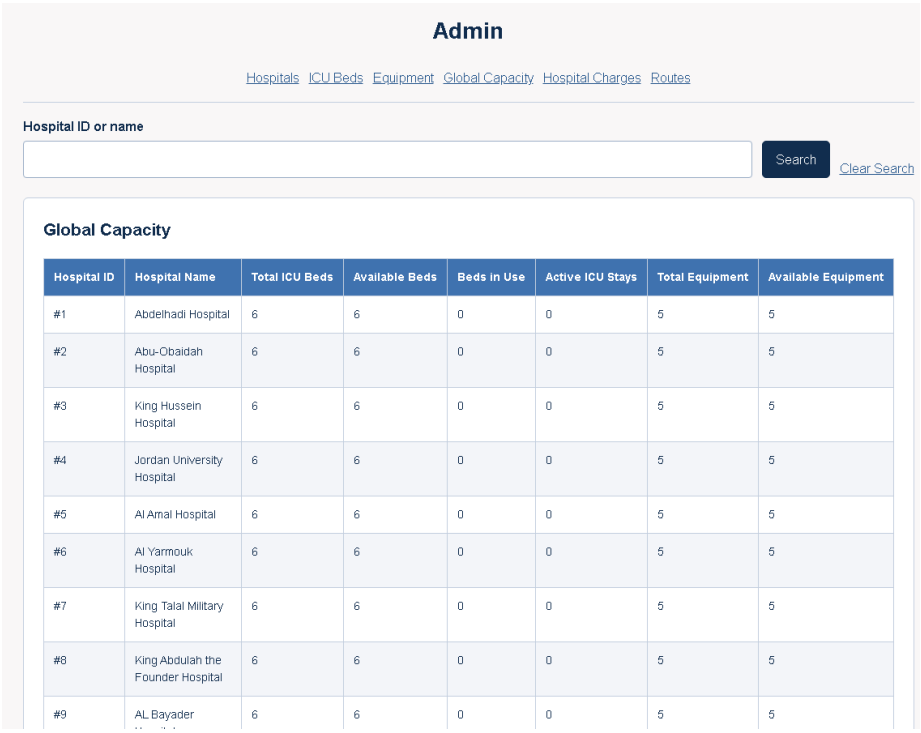


Figure 72: Global Capacity

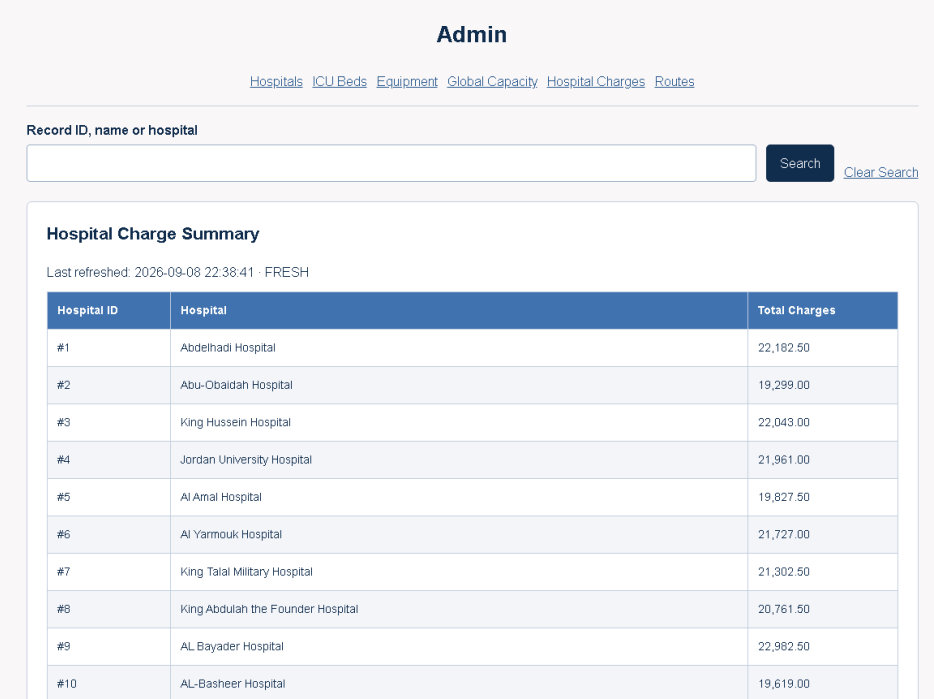


Figure 73: Charges Per Hospital

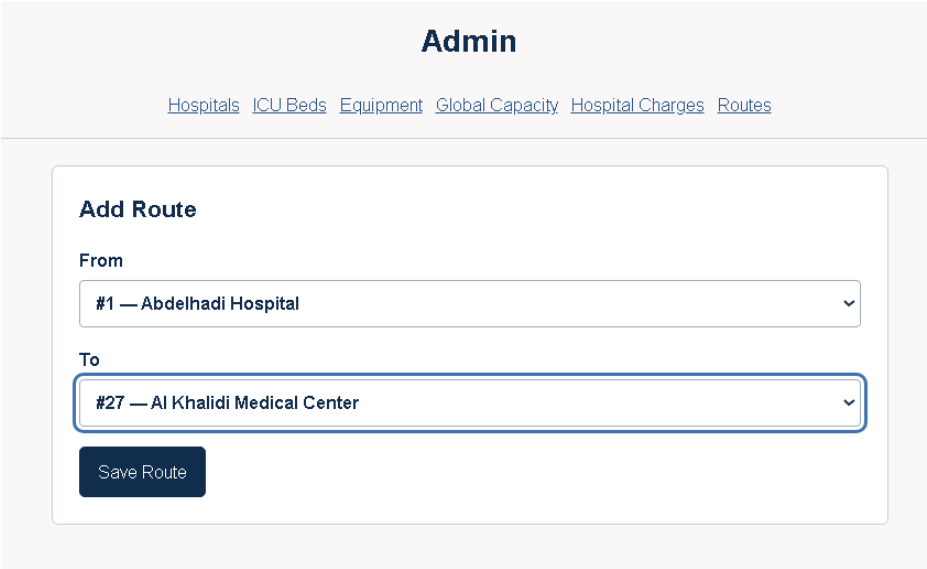


Figure 74: Adding a Route

4.1.2 User

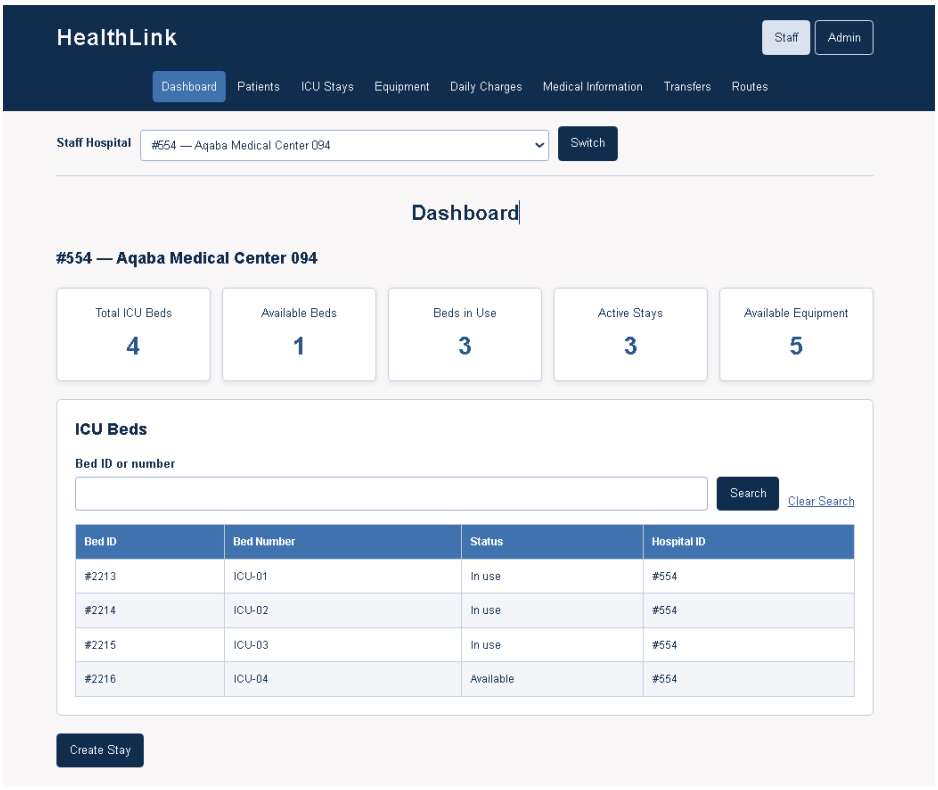


Figure 75: User Homepage

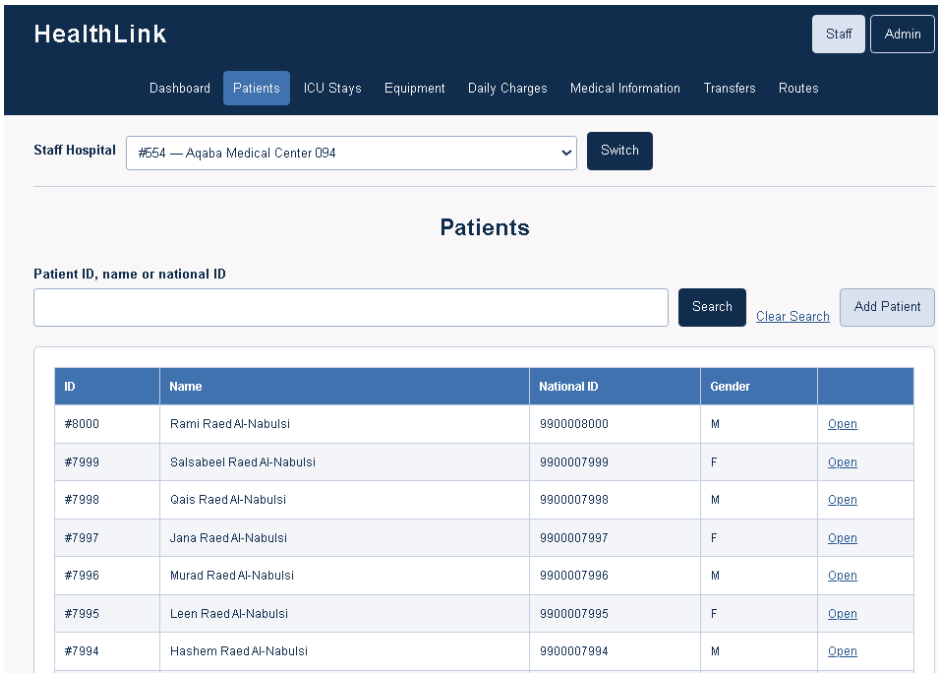


Figure 76: Patient of User's Hospital

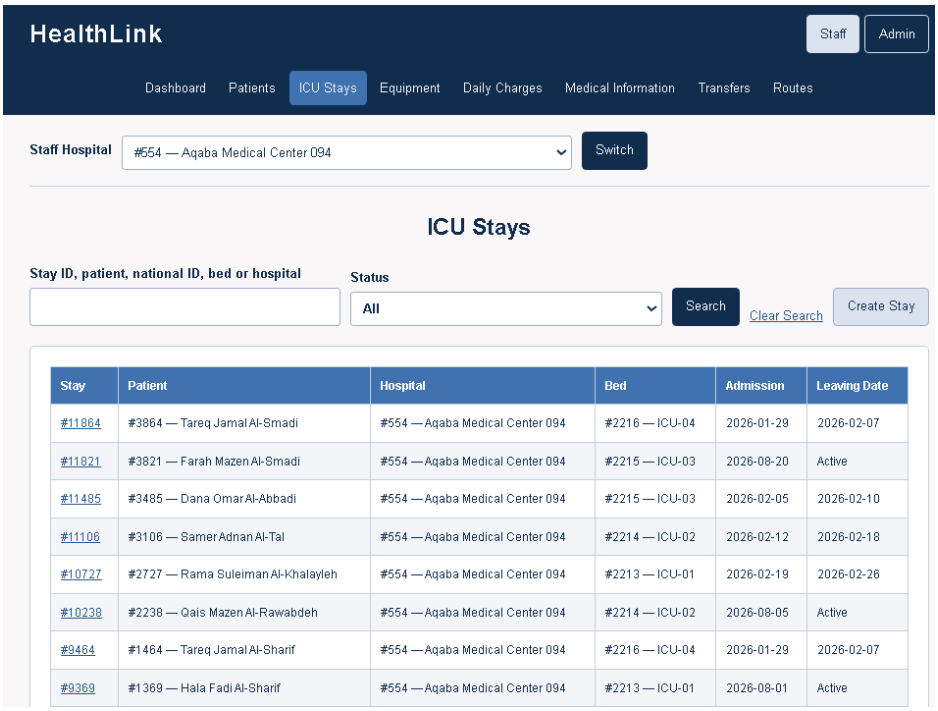


Figure 77: Stays of User's Hospital

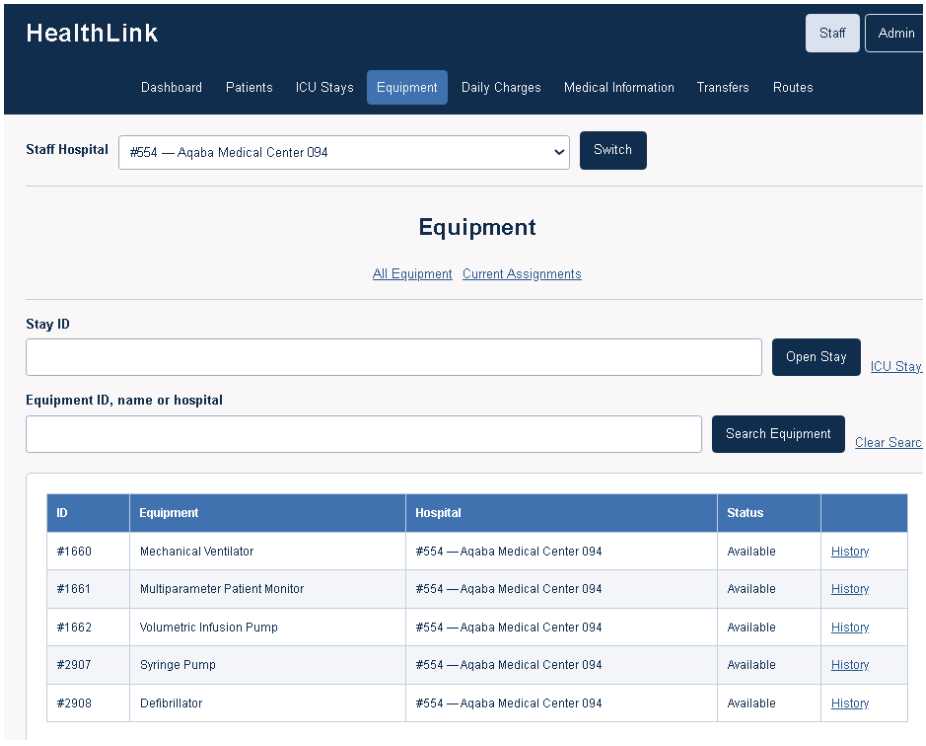


Figure 78: Equipment of User's Hospital



Figure 79: Charges of User's Hospital

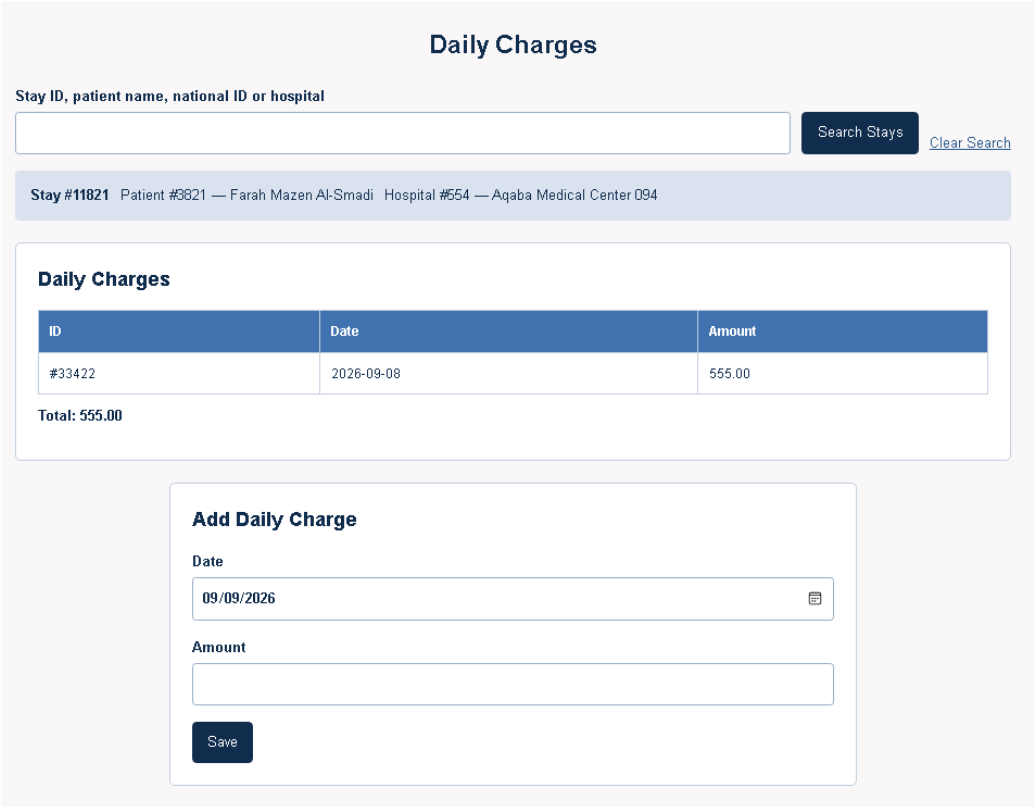


Figure 80: Daily Charges of Selected Stay

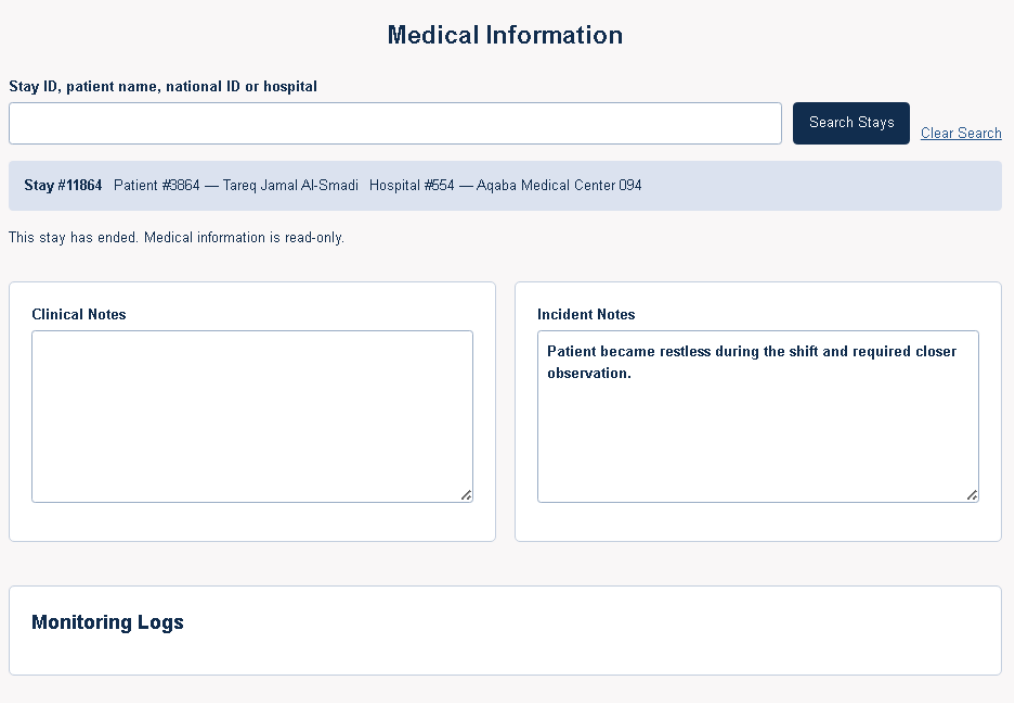


Figure 81: Medical Information of Selected Stay

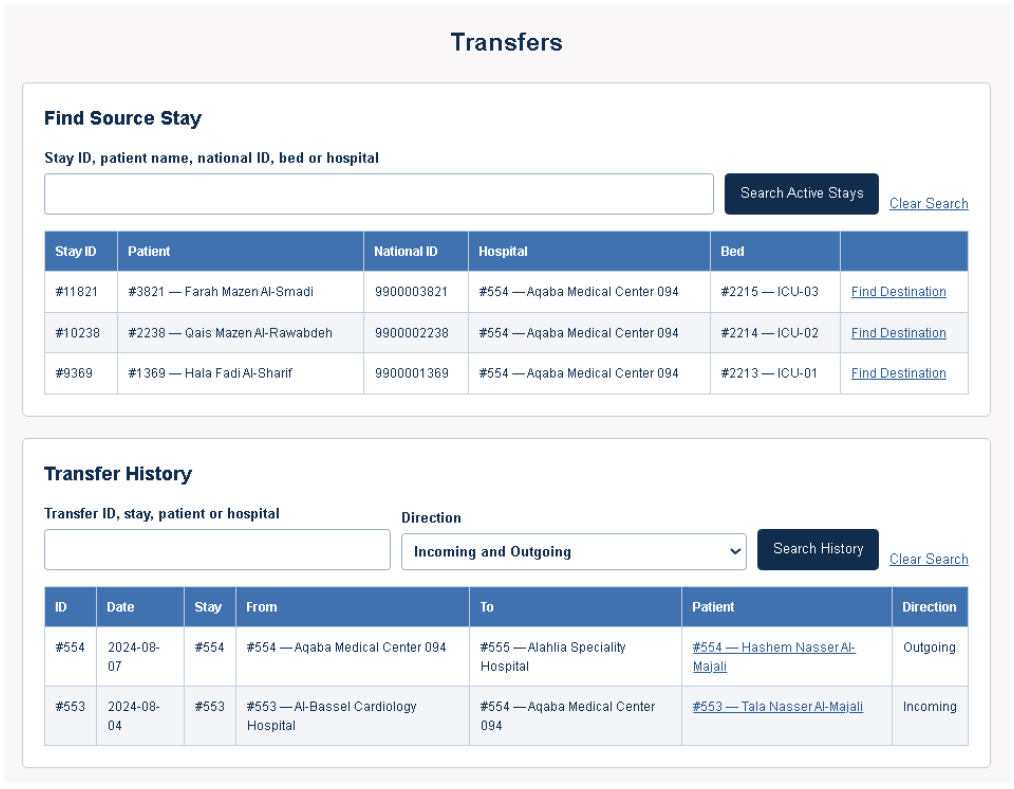


Figure 82: Transfers of User's Hospital

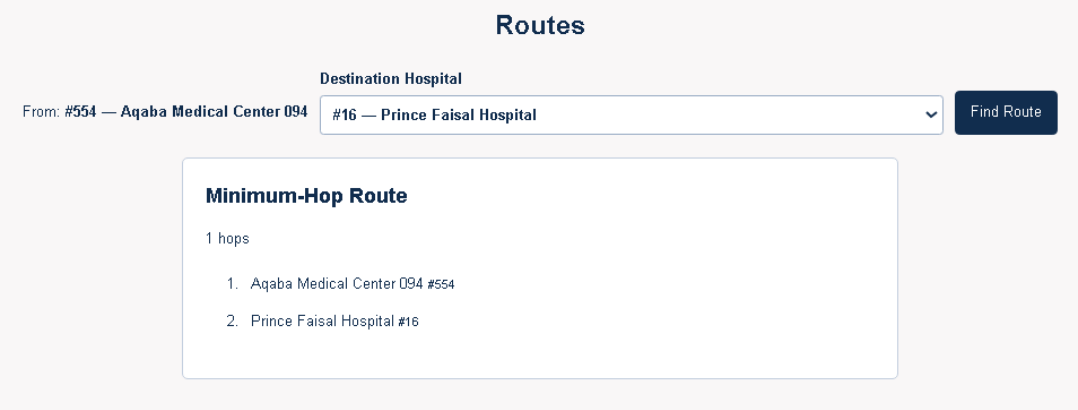


Figure 83: Routes from User's Hospital

4.1.3 Running Core Transactions

Package Usage

```
$sql = "BEGIN billing_PKG.addBill_PRC(:stayId, TO_DATE(:chargeDate, 'YYYY-MM-DD'),
        TO_NUMBER(:amount, '99999999D99', 'NLS_NUMERIC_CHARACTERS=''.,'''')); END;";
```

Figure 84: Package Usage

Here the code calls billing_PKG.addBill_PRC when the Staff adds a bill. It sends the stay ID, date and amount to the package. The package adds the bill and commits it.

Procedure Usage

```
$sql = "BEGIN finishStay_PRC(:stayId, TO_DATE(:leavingDate, 'YYYY-MM-DD')); END;";
```

Figure 85: Procedure Usage

Here the code calls finishStay_PRC when the Staff enters the leaving date. It sends the stay ID and leaving date to the procedure to finish the stay.

View Usage

```
if ($hospitalId === 554) {
    $sql = 'SELECT bedId, bedNumber, status, hospitalId
    FROM hospitalBeds_view
    WHERE hospitalId = :hospitalId';
```

Figure 86: View Usage

Here, if the hospital ID is 554, the code gets the beds from hospitalBeds_view. If it is another hospital, it gets them from ICU_Bed.

Materialized View Usage

```
if ($hasChargesView) {
    $sql = 'SELECT hospitalId, hospitalName, "SUM(D.AMOUNT)" AS totalCharges
        FROM hospitalCharges_MatView
```

Figure 87: Materialized View Usage

Here the Admin code gets the hospital charge totals from hospitalCharges_MatView.

4.2 Testing

4.2.1 Successful Run Test

Test 1: Adding Medical Information to a Stay

This test will add medical information to ICU stay using stayId = 9369.

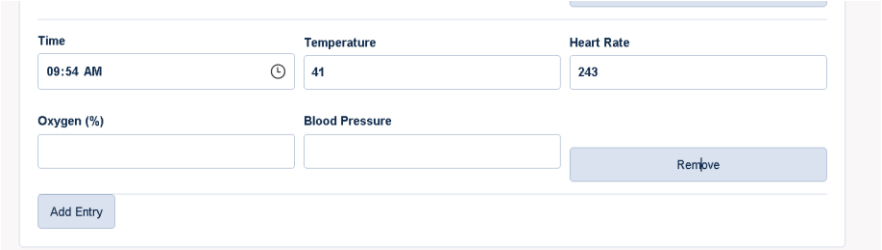


Figure 88: Adding Medical Information

Now we will check Oracle and neo4j to make sure that the information was saved using the correct stayId.

	S...	PATIENTID	BEDID	ENTERINGDATE	LEAVINGDATE
1	11821	3821	2215	20-AUG-26	09-SEP-26

Figure 89: Validate Stay ID

```
{
  _id: ObjectId('6a9882d46133bf68f21326d5'),
  stayId: 9369,
  monitoringLogs: [
    {
      time: '00:00',
      temperature: 39,
      bloodPressure: '142/81'
    },
    {
      time: '09:54',
      temperature: 41,
      heartRate: 243
    }
  ],
  incidentNotes: [
    'Patient reported sudden dizziness and was reassessed.'
  ]
}
```


Figure 90: Validate Added MongoDB Data

As we can see, ICU stay exists in Oracle and the medical information was added to MongoDB to the same stay which shows that the databases are connected correctly using the stayId.

Test 2: Completing a Transfer

This test will complete a transfer using stayId = 11821

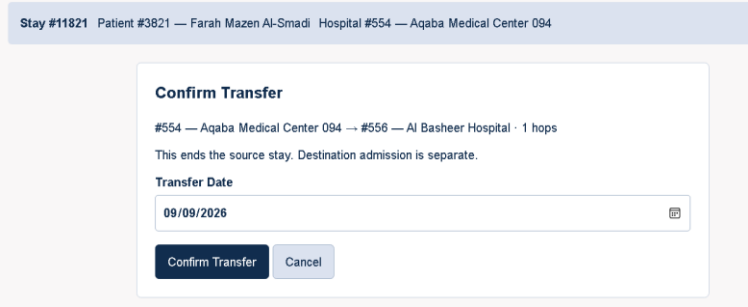


Figure 91: Completing a Transfer

Now we will check Oracle and neo4J to make sure that the transfer was saved in both databases.

TRANS...	TRANSFERDATE	STAYID	FROMHOSPITALID	TOHOSPITALID
701	09-SEP-26	11821	554	556

Figure 92: Validate Transfer in Oracle

h1	h2	r.stayId	r.transferDate
Aqaba Medical Center 094	Al Basheer Hospital	11821	"2026-09-09"

Figure 93: Validate Transfer in Neo4j

As we can see Oracle and Neo4j added the transfer successfully to its database which shows that the transfer data was correctly saved in both systems.

4.2.2 Invalid Input Test

Test 1: Adding Invalid Daily Charge

This test will try to add a negative amount to daily charge.

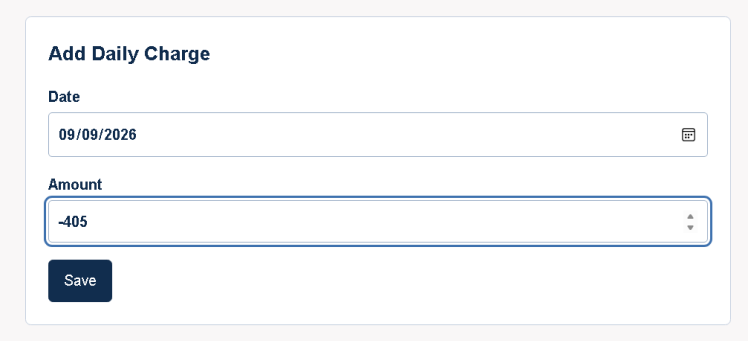


Figure 94: Adding Invalid Amount

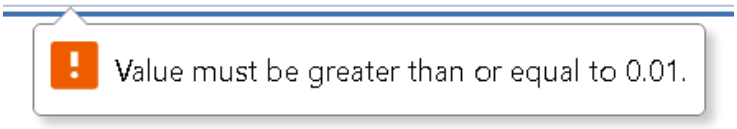


Figure 95: Result in Application

CHARGEID	CHARGEDATE	AMOUNT	STAYID
28105	01-AUG-26	203.25	9369
28106	02-AUG-26	329.5	9369
28107	03-AUG-26	467.75	9369

Figure 96: Result in Oracle

As we can see the negative amount was rejected and it wasn't added to Oracle database.

4.2.3 Error Test

Test 1: MongoDB Connection Error

This test will simulate a connection error to MongoDB while adding medical information to an ICU stay, as I will terminate the connection and try to add medical information.

I will open the page where I can add medical information for a stay with stayId = 10238.

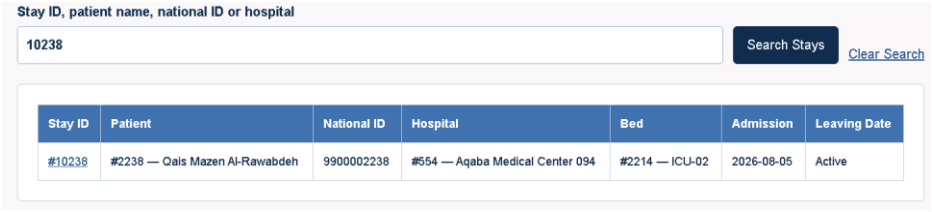


Figure 97: ICU Stay

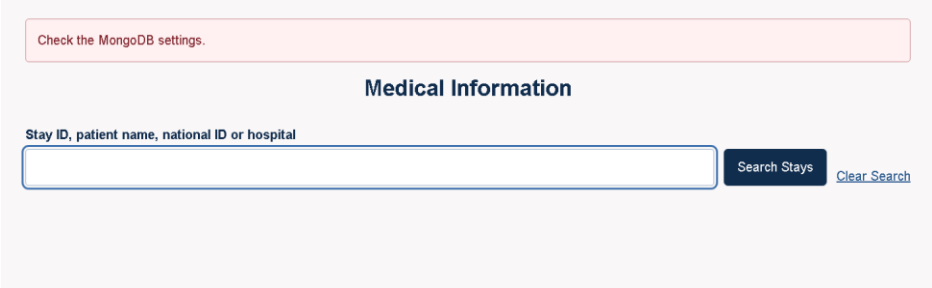


Figure 98: Failure

As we can see, we couldn't even open the page where I can add medical information and it showed a message saying, "Check the MongoDB settings".

4.3 System Architecture Evaluation

My project uses Oracle, MongoDB and Neo4j in one system, but each database has a different responsibility. Oracle is the main database because it stores the hospital records that must remain accurate. It stores the hospitals, ICU beds, patients, ICU stays, equipment, daily charges and transfers.

The data starts in Oracle when staff selects an ICU stay. The application checks the stayId and gets the stay information from Oracle. When staff adds medical information, the application uses the same stayId to save the clinical notes, monitoring logs and incident notes in MongoDB. This means that the medical document in MongoDB is connected to the ICU stay stored in Oracle. The transfer flow also starts in Oracle. The application checks the active stay and identifies the source hospital through its ICU bed. Neo4j is then used to search for a route to the destination hospital. After the transfer is confirmed, Oracle ends the original stay and stores the transfer record. Neo4j stores the historical transfer relationship between the hospitals.

Criteria	Oracle	MongoDB	Neo4j
Data Accuracy	Oracle is more accurate for the main records because it checks its own rules	MongoDB is less strict for the connection with Oracle because it only stores the medical	Neo4j is accurate for the route and transfer relationship stored inside Neo4j. It does

	before saving the data. It can reject negative charges, duplicate charges, unavailable beds and equipment conflicts.	document. It does not automatically check whether the stayId exists in Oracle, so the PHP code must do this check.	not automatically check whether the stayId or transferId exists in Oracle, so the PHP code must do this check.
Speed	Oracle is slower than the other two for some operations because it checks tables, relationships, constraints, triggers and transactions.	MongoDB is faster for medical information because it can directly read or save one document using stayId.	Neo4j is faster for route searching because it follows the hospital relationships directly instead of using several Oracle joins.
Latency	Oracle adds time because the request includes the query, validation and transaction processing.	MongoDB adds another request after the Oracle stay check, so the page must wait for another database response.	Neo4j adds the most network delay in this case because the PHP page sends a request to the Neo4j HTTP endpoint and waits for the route result.

Oracle is the most accurate database in my project because the main hospital records require strict rules. For example, the same bed should not be assigned to two active stays, unavailable equipment should not be assigned, and a negative daily charge should not be inserted. MongoDB is useful for the medical information because the information can be stored as one flexible document. However, the connection between MongoDB and Oracle depends on the application checking the stayId. MongoDB could contain incorrect information if the application saved a document without checking whether the stay existed in Oracle. Neo4j is useful for route searching because hospitals are connected through relationships. Finding the best route is more suitable in Neo4j than using many relational joins. However, Neo4j is updated separately from Oracle.

Oracle uses a stronger ACID approach for the main hospital data. This means that the Oracle transaction is completed fully or rejected, the data must remain valid, separate transactions should not interfere incorrectly and committed data remains saved. The complete project is not one ACID transaction because Oracle, MongoDB and Neo4j are updated separately. If Oracle saves a transfer but Neo4j fails, the transfer will exist in Oracle but the historical relationship will be missing in Neo4j. This creates a BASE problem between the databases because the data may only become consistent later.

The speed is different because each database handles a different type of operation, as MongoDB can quickly load the medical document, and Neo4j can quickly search the hospital relationships. But oracle may take more time because it checks more rules, but this processing is needed to protect the accuracy of the main hospital data. The latency is higher when one operation contacts more than one database. A transfer may need an

Oracle request to check the stay, a Neo4j request to find the route, another Oracle request to save the transfer and another Neo4j request to save the historical relationship. The total response time includes all these requests, database connections and network delays.

The testing was effective for checking the main functions of my project. It showed that the Oracle rules were working because incorrect charges, duplicate charges, unavailable beds and equipment conflicts were rejected. It also showed that MongoDB and Neo4j were connected to the Oracle data by using the same stayId. The testing was good for checking the normal use of the system. It checked if the databases could perform their own tasks and if the data was being connected between the databases. It also showed that Oracle was protecting the main hospital records.

But it did not show what happens when one database works and another database fails, for example, we could create a new transfer, Oracle saves it, but Neo4j could fail to save the history. Also, it did not show if MongoDB fails after Oracle checking that the stay id exists. Another thing is the testing was manual, so, the results depend on the tester being able to follow the steps correctly. Also, I could not test every possible error or case, therefore, some problems may not have been noticed.

References

1. Composite Unique Constraint SQL - Stack Overflow
2. PostgreSQL 15 Security Invoker Views: RLS… | JusDB Blog
3. vldb.org/conf/1997/P136.PDF
4. Incremental Materialized View: How to Keep Derived State Fresh in Real Time | Tacnode Blog
5. CURRENT_DATE
6. Stored Procedures vs. Application Logic: Where Should Business Rules Live?
7. Domain Logic and SQL
8. How to Export Data using SQL Developer
9. sqlplus - Oracle query execution time - Stack Overflow
10. mongodb: how can I see the execution time for the aggregate command? - Stack Overflow
11. ResultSummary (Neo4j Java Driver 6.2 API)
12. https://neo4j.com/docs/query-api/current/query
13. InfoResCom-1-2-104.pdf
14. Data consistency models: ACID vs. BASE databases explained